AutoTune AI - Progress Checkpoint

2 November 2026 | Documentation-only release v0.8.5 (source remains v0.8.3)

Current position. We have moved from CPU-only synthetic self-healing to genuine PyTorch and Hugging Face LoRA training. Adapter integrity, fresh reload, and training continuation to completion were user-verified on Windows. The complete real-model agent-driven retry loop is not yet verified.

At-a-glance evidence

Latest Windows tests	Model verification	Resume milestone
192 passed, 1 skipped; 22.26 s	Tiny GPT-2 PEFT LoRA; 128 trainable parameters; 6 adapter + 10 checkpoint files checked	Pause at step 2; explicit resumed process completed 4/4 steps

What has been accomplished

• Phase 1: configuration contracts, fingerprints, environment doctor and automated tests.

• Phase 2: metric events, overfit/underfit/NaN/plateau and gradient monitoring, offline replay.

• Phase 3: persistent mock recovery journal, checkpoint hash, retry-budget and crash-safety tests.

• Phase 4: bounded Meta advisor contract, safe policy-checked hyperparameter patches (mock tested).

• Phase 5A: genuine CPU PyTorch small language-model training and loadable full-weight checkpoint.

• Phases 5B-5D: Hugging Face CPU LoRA fine-tuning, read-only SHA audit, fresh adapter generation, paused-run continuation.

• Additional: CPU text-classification backend (offline tested); Unsloth accelerator backend (GPU preflight only).

Evidence limitations

The demonstration dataset has only 4 training rows and 2 validation rows. Recorded best validation loss is 10.365182876586914, which is not a useful quality benchmark. The source can produce a resume proof, but its exact optimizer and scheduler contents have not been shared or independently reviewed. No hosted Meta call, MLflow tracking server, deployed Kubeflow/Katib cluster, or live Unsloth GPU fine-tune has been verified.

Delivery status by workstream

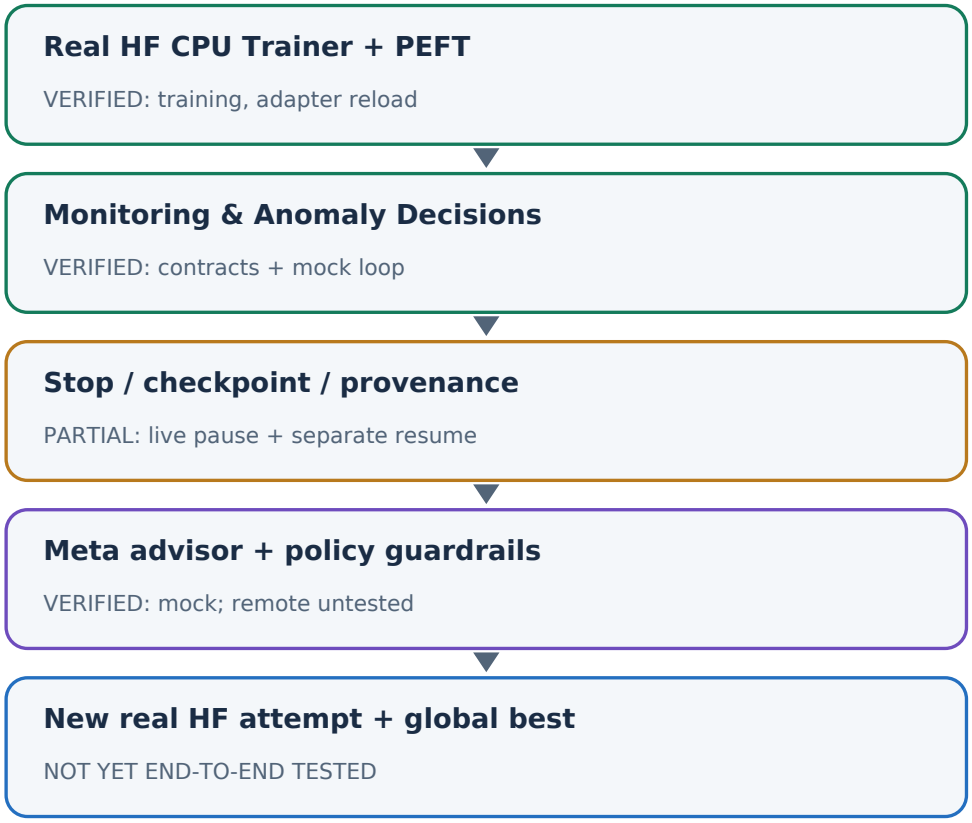
Status meanings: VERIFIED = user output or local tested behavior; PARTIAL = one portion is verified; PENDING = no end-to-end implementation proof.

Workstream	Evidence / scope	State
Foundation & diagnostic engine	15, 36 tests and simulated loss monitoring	VERIFIED
Durable CPU-mock recovery	48 tests, atomic journal, synthetic checkpoint	VERIFIED (MOCK)
Meta diagnostic advisor	77 tests, mock patch, no hosted API verification	MOCK VERIFIED
Real PyTorch CPU	95 tests; GRU smoke with .pt weights	VERIFIED
HF PEFT LoRA	User Tiny GPT-2 run; validation loss + final adapter	VERIFIED
HF adapter audit / reload	6 adapter files; 10 checkpoint files; 8 generated tokens	VERIFIED
HF checkpoint continuation	Step 2 stop -> new process -> step 4 completed; inspect resume proof	PARTIAL / PROOF REVIEW
CPU BERT classifier	Implementation + offline contracts; no user live run	OFFLINE TESTED
Unsloth training	GPU config preflight, CPU training blocked	GPU RUN PENDING
Agentic real-HF retry	No live two-attempt recovery experiment	PENDING
MLflow / Kubeflow / Katib	Examples/hooks exist; no deployed proof	PENDING
Git updates / Dashboard / Evaluation	Production end-to-end workflow still to be built	PENDING

Latest test checkpoint: 192 passed, 1 skipped on the user's Windows installation. Earlier build environment: 193 passed. The skipped case must not be misreported as passing.

Next integration: real self-healing

The architecture below marks the gap between verified reusable components and the unverified complete training/retry cycle.



Bounded retries, idempotent receipts, fail-closed recovery, untouched holdout set

Design guardrails

• A model recommendation is data, never executable code. Reject arbitrary commands and unapproved configuration fields.

• When learning rate, optimizer, or LoRA module layout changes, start a new trial unless full-state checkpoint compatibility is proven.

• Verify checkpoint hashes and training receipts before continuing or considering a trial result.

• Limit model calls, training steps, retries and resource usage; retain all training and validation provenance.

• Select candidates using validation data. Keep a separate untouched holdout set for independent evaluation.

Next 5E sprint - concrete steps

Priority: integrate the verified Hugging Face CPU LoRA path with the existing deterministic monitor, policy engine, and durable recovery state machine.

Order	Implementation task	Acceptance evidence
0	Inspect existing resume_proof.json; do not retrain	begin=2; end=4; optimizer state nonempty; scheduler state restored
1	Standardize paired train/eval metric windows from real HF callback	Timestamp/step-consistent MetricEvent and replay test
2	Connect real trainer to durable state and artifact receipts	Verified model/optimizer/adapter/checkpoint hashes; crash-safe journal
3	Integrate Meta advisor mock + bounded policy patch	Strict schema, rejected advice fallback, versioned new trial config
4	Run automatic two-attempt CPU training scenario	First diagnosed stop; separate new attempt; best validation candidate selected
5	Fault injection + Windows verification	Crash, mismatch, tampering, retry exhaustion and duplicate-launch protection
6	Add local MLflow tracking; then plan CPU Kubeflow/Katib	Inspect experiment lineage and artifacts in running local server

Exact next user command

From the existing AutoTune AI folder in VS Code / Windows PowerShell:

```
Get-Content outputs\gpt2-resume-probe-01\resume_proof.json -Raw
```

Project continuity

The existing v0.8.4 handbook (49 pages) follows this supplement in full. AGENTS.md and the master plan carry a versioned summary at the top while preserving the historical log. This v0.8.5 delivery changes documentation only; it does not claim additional training, tests, or new source code.

Checkpoint v0.8.4
Real CPU Resume Verified

2 November 2026 | Evidence update only; source release remains v0.8.3

The Windows CPU test exercised an actual interrupted Hugging Face PEFT training attempt, resumed it from checkpoint-2, and finished the scheduled four steps. The Unsloth preflights verified configuration handling, not GPU execution.

What was verified on the user machine

System check	Observed evidence	Interpretation
pytest suite	192 passed; 1 skipped; 22.26 s	Current code package passed CPU checks; reason for skip not established.
Controlled training pause	status=paused, global_step=2, target_step=4	The first process saved checkpoint-2 and stopped deliberately.
Checkpoint continuation	Resume command completed; ok=true; output shows step 4/4	Training continued from the specified path.
Best validation loss	10.365182876586914	Toy 4/2 dataset; not a quality benchmark.
Unsloth GPU configuration	ok=true; executable_on_configured_device=true; live_gpu_training_verified=false	Configured target is a GPU; actual device was not verified.
Unsloth CPU configuration	ok=true; executable_on_configured_device=false	Correctly declines CPU Unsloth accelerated training.

Evidence still awaiting review

The CLI produced outputs/gpt2-resume-probe-01/resume_proof.json, but the user has not shared its contents. The implementation requires nonempty optimizer state and a sufficiently advanced scheduler on resume. Inspect the recorded values before marking independent optimizer/scheduler state evidence fully reviewed.

The reported best checkpoint remains checkpoint-2; the later steps completed successfully but did not necessarily improve the lowest recorded evaluation loss.

Evidence Review and Interpretation

Step 1 - Inspect the saved resume evidence

Windows PowerShell, from the project root:

```
Get-Content outputs\gpt2-resume-probe-01\resume_proof.json -Raw
```

The resume proof is designed to record the following checks:

Field	Expected evidence
ok	true
checkpoint_step	2
trainer_begin_step	2
trainer_end_step	4
optimizer_state_nonempty_at_resume	true
scheduler_last_epoch_at_resume	at least 2

What the trainer does on resume

The Python implementation checks the manifest and matching training plan before invoking Trainer.train(resume_from_checkpoint=...). Its on_train_begin callback reads the restored optimizer and learning-rate scheduler before executing further optimization steps. It refuses a missing optimizer state, a mismatched global step, or a scheduler behind the paused step.

Interpreting the result correctly

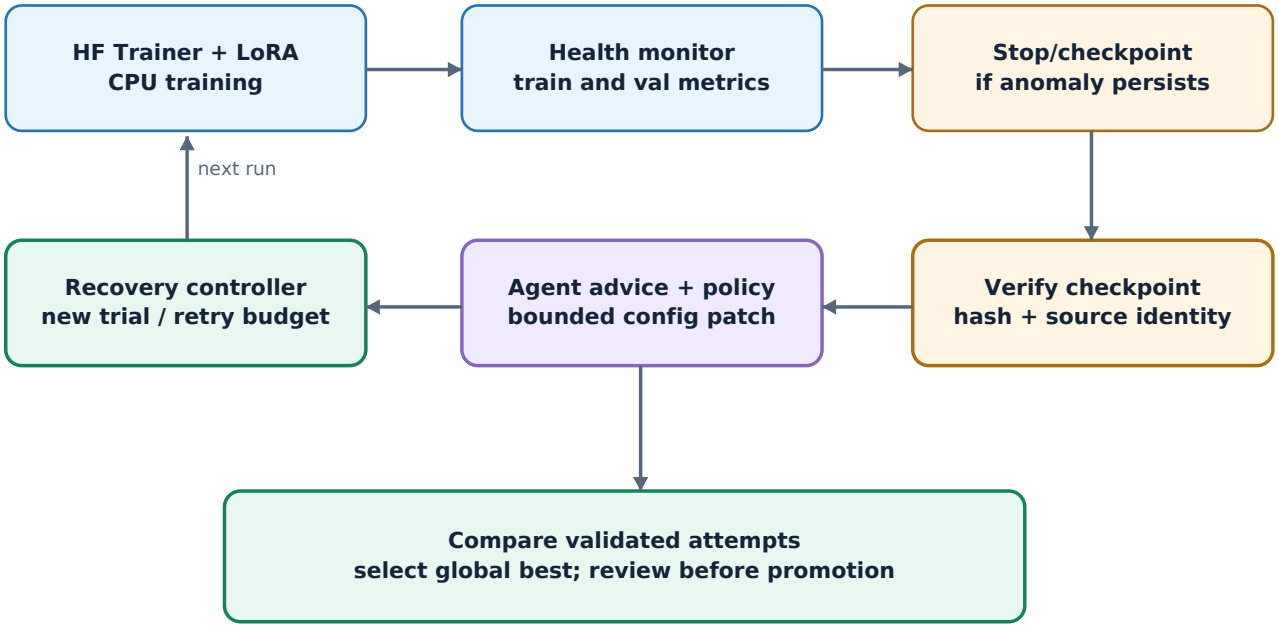
A successful finish is strong operational evidence of a stateful continuation because the resume callback has protective assertions. It is still not a bitwise equivalence experiment against uninterrupted training; hardware, nondeterministic kernels and sampling may affect exact values. No full self-healing retraining cycle was executed in this test.

Step 2 - Keep run folders immutable

Do not modify either outputs/gpt2-cpu-01 or outputs/gpt2-resume-probe-01. For a new autonomous recovery attempt, create a separate named run folder, preserve its original hashes, and write any policy-approved configuration change to a new versioned attempt configuration.

Next Architecture: Phase 5E

Connect real Hugging Face training to the existing monitor, diagnostic advisor, durable controller and policy engine. Preserve the CPU Hugging Face PEFT path as default and Unsloth as optional GPU-only backend.



Suggested Phase 5E flow. Not yet an end-to-end verified real-HF retraining loop.

Role boundaries

Component	Core responsibility
HF Trainer / PEFT	Train LoRA adapters, emit callbacks, checkpoint full Trainer state.
Metric monitor	Use multiple train/validation signals and minimum sample counts to detect persistent unhealthy patterns.
Meta diagnostic advisor	Propose reasons and bounded parameter edits; offline fallback must always work.
Policy + recovery controller	Check value ranges, run budget, provenance, retry count and checkpoint compatibility.
Validation selector	Compare candidates using validation only; untouched holdout reserved for final evaluation.

Phase 5E - Implementation and Acceptance Plan

Implement in small, independently testable steps

1. Introduce a uniform trainer adapter interface emitting evaluation events, stop intent, checkpoint metadata and completed result receipt for real HF causal-LM LoRA.

2. Connect HF evaluation callback to Phase 2 MonitorStream, with robust signal quality and minimum evidence before diagnoses.

3. Let Phase 3 durable controller own attempt states, atomic receipts, checkpoint provenance and restart decisions; avoid double-run on interruptions.

4. Call the existing Meta advisor mock or deterministic fallback to propose a bounded YAML patch. Do not let the model execute arbitrary code.

5. Create a separate trial directory and versioned configuration; resume only when the checkpoint and optimizer configuration are compatible. Otherwise start a new training attempt.

6. Run a CPU end-to-end test with a deliberately constructed loss pattern; label it synthetic-triggered if overfitting is artificially induced.

7. Run a non-artificial small-model experiment with real metrics and evaluate whether any recovery produces a real validation improvement; do not guarantee improvement.

Completion gates

Gate	Required evidence
Functional	Real HF attempt writes metrics, stops safely, records diagnosis and starts another separate attempt.
Safety	Invalid advice, tampered checkpoints, schema mismatch and exhausted retry budgets refuse to launch.
Repeatability	Crash/relaunch does not duplicate attempts; manifests and provenance remain verifiable.
Evaluation	Select best valid checkpoint by validation; do not claim test-set improvements from toy splits.

Deferred: MLflow tracking, CPU Kubernetes/Kubeflow, Katib trials, UI and remote GPU Unsloth. No hosted Meta API or actual Unsloth GPU run is verified. The inherited 45-page handbook follows this current checkpoint supplement.

ENGINEERING HANDBOOK | LIVING EDITION

AutoTune AI

An Agentic, Self-Healing MLOps Platform

Architecture, training diagnostics, guarded hyperparameter recovery, Kubeflow orchestration, Meta advisor integration, implementation roadmap and verified baseline.

Version: v0.8.2 | Checkpoint: Real Tiny GPT-2 CPU LoRA success + artifact audit gate | 01 November 2026

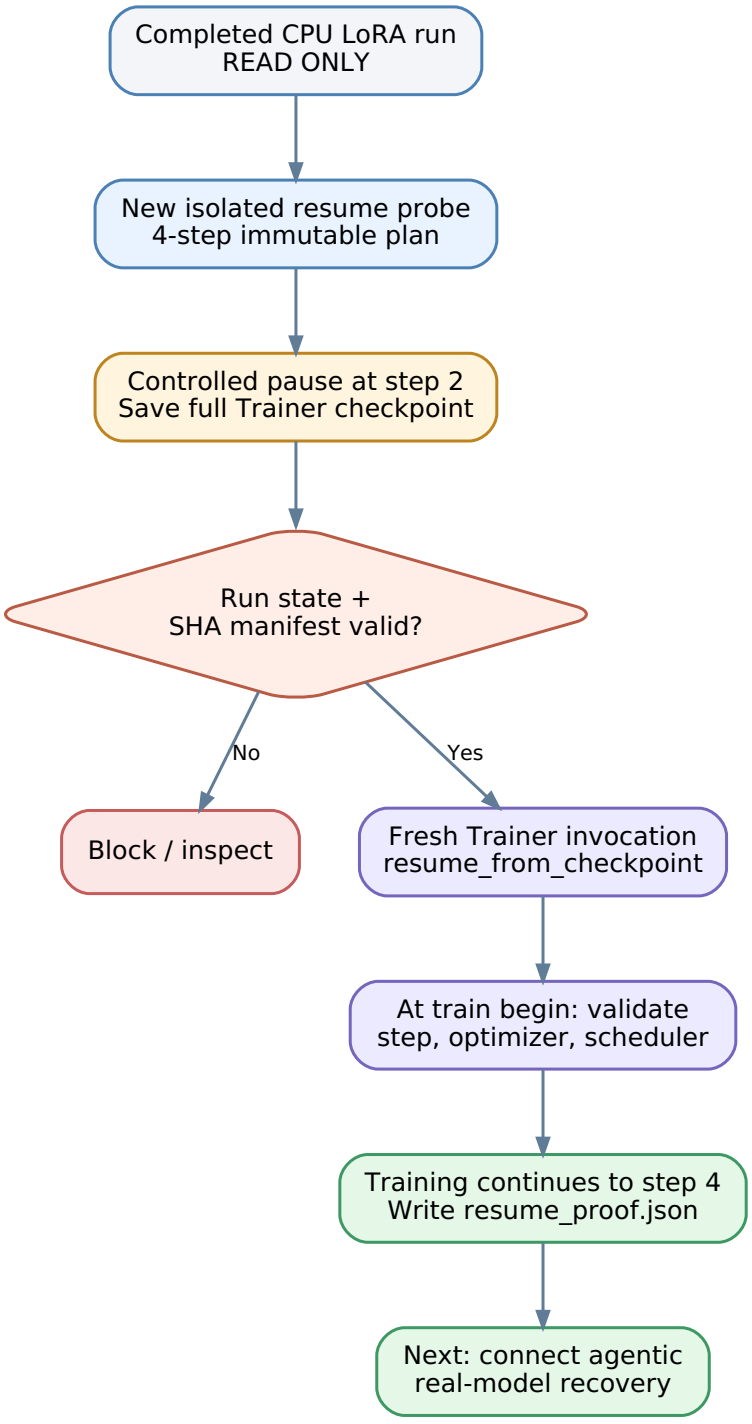
This handbook distinguishes CPU synthetic recovery from genuine small-model PyTorch training. Real tiny GPT-2 CPU LoRA training was user-verified. No hosted Meta API, Meta Llama GPU fine-tuning, optimizer resume, or production deployment is claimed.

v0.8.2 - Completed artifact checks and next real checkpoint-resume exercise

> **User-confirmed Windows verification (2026-11-01):** The existing CPU LoRA run passed the read-only integrity audit of **six final-adapter files** and **ten full-checkpoint files**. A fresh base-model + LoRA adapter reload completed and generated **eight tokens**. The result reported best validation loss **10.365182876586914**. Neither audit resumed training: both explicitly reported `actual_resume_executed: false`. These are user-supplied terminal results, not executed in the document-build environment.

Why a separate resume exercise is necessary

The original completed output `outputs/gpt2-cpu-01` must remain read-only. A final adapter reload proves inference, but not continuation of the Trainer optimizer. Phase 5D introduces a **new** immutable four-step experiment: intentionally pause after step two, save complete Trainer state, verify hashes and run-state records, then invoke Trainer in a new process to continue from that same checkpoint.



Controlled and auditable CPU Trainer checkpoint-resume workflow

Implementation contracts and acceptance evidence

Boundary	Required behavior
Fresh probe	New output only; identical tiny public GPT-2 base and data
Pause gate	Step must be positive, aligned with evaluation/save, and less than total 4
Checkpoint	SHA-256 manifest covers adapter, optimizer.pt, scheduler.pt, trainer_state.json, RNG
Resume gate	Only a verified `paused` run with matching config/data/plan/path/step

Boundary	Required behavior
Training state	At train begin, observe restored step 2, nonempty optimizer state and scheduler progress
Completion	Continue past the old global step; write `resume_proof.json` and final adapter
Failure	Enter blocked state; no silent retry or in-place overwrite of completed reference

Run the two commands from the extracted project root in VS Code PowerShell; a public model download may occur if the cache is cold:

```
.\.venv\Scripts\python.exe -m autotune.hf_models_cli --config config/lora_cpu_resume_probe.yaml
        --execute --pause-after-step 2 --output outputs/gpt2-resume-probe-01
.\.venv\Scripts\python.exe -m autotune.hf_models_cli --config config/lora_cpu_resume_probe.yaml
        --execute --resume-checkpoint outputs/gpt2-resume-probe-01/checkpoints/checkpoint-2 --output
        outputs/gpt2-resume-probe-01
```

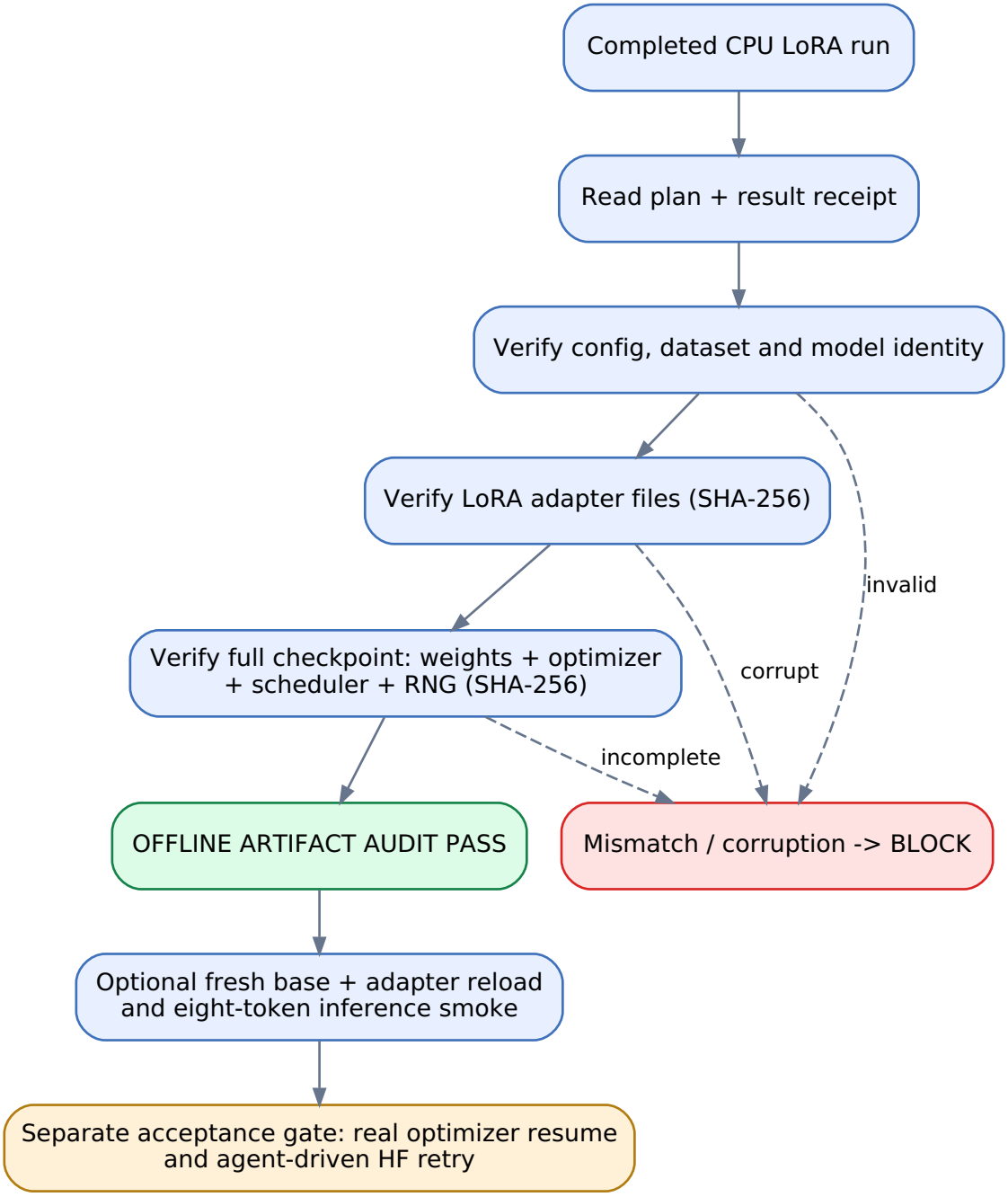
Inspect `outputs/gpt2-resume-probe-01/resume_proof.json`. The intended successful evidence is `trainer_begin_step: 2`, `trainer_end_step: 4`, `optimizer_state_nonempty_at_resume: true`, and `scheduler_last_epoch_at_resume >= 2`. **This outcome has not yet been observed**. It is a planned user-machine acceptance test; no live Trainer resume was executed in the build environment.

Current test scope: 185 offline tests passed (166 inherited, 19 new). These validate the pause/resume contracts and CLI wiring without requiring Hugging Face runtime dependencies. They do not verify training quality, numerical reproducibility or live optimizer restoration. See `docs/PHASE5D_OPTIMIZER_RESUME.md` for the operator runbook.

Latest verified evidence - v0.8.2

> **Independent user terminal checkpoint (2026-11-01):** Real PEFT LoRA fine-tuning of public `sshleifer/tiny-gpt2` succeeded on Windows CPU. The user reported one epoch, two training steps, four training and two validation examples, training loss about 10.4 and best validation loss **10.365182876586914**. There were **128 trainable LoRA parameters**, a saved final adapter, and a best full Trainer checkpoint at `outputs/gpt2-cpu-01/checkpoints/checkpoint-2`. These facts establish that the small-model training and saving path ran, not that the model learned useful language quality. Adapter reload was subsequently user-verified; optimizer resume and a real automatic recovery retry remain unverified.

New read-only audit and reload gate



Verifier for the first real CPU LoRA result

The audit first validates the completed run's provenance (`plan.json`, dataset/config hashes), then checks the final adapter SHA-256 manifest, and separately checks that the full Trainer checkpoint has hashed adapter, optimizer, scheduler and RNG files. It refuses files outside the run, mismatching receipts, incomplete results and changed datasets. A **successful file-integrity audit is not the same as actually resuming the optimizer**. Optional `--reload` freshly

constructs the base model, applies the saved adapter and generates eight deterministic tokens; this is a load/inference test, not a benchmark.

From the exact project directory and existing `.venv` in VS Code PowerShell:

```
.\.venv\Scripts\python.exe -m autotune.hf_audit_cli --config config/lora_public_cpu.yaml --run outputs/
        gpt2-cpu-01
.\.venv\Scripts\python.exe -m autotune.hf_audit_cli --config config/lora_public_cpu.yaml --run outputs/
        gpt2-cpu-01 --reload
```

The second line is optional and may download the base weights; run it only after the offline audit passes. Do not try to resume the already completed output in place. A dedicated interrupted-run exercise is the next critical training-integrity test.

Verification scope: 166 automated tests pass in the local build environment (153 inherited, 13 new artifact-audit cases), including tampered adapter, optimizer file corruption, mismatched model/data/config, path escape, interrupted run refusal and mocked reload integration. There is **no live HF inference run in this build environment**. Read `docs/PHASE5C_ARTIFACT_AUDIT.md` for implementation details and exact operations.

Executive overview

AutoTune AI is a constrained feedback controller for ML training and fine-tuning. Instead of relying on a human to constantly inspect loss curves, it observes training events, detects potential generalization or numerical issues, preserves a known-good candidate and starts a new **bounded** experiment with verified parameters. A Meta model may advise the diagnosis and hyperparameters, but it cannot directly execute code or rewrite arbitrary files.

Design promise: *autonomous experiments; deterministic guardrails; auditable evidence; explicit release control.*

> **Historical v0.7 evidence:** The user verified Phase 5A real PyTorch tiny-GRU training and 95 passing tests on Windows. New Phase 5B offline contracts, LoRA source pipeline and checkpoint manifest tests raise build-container coverage to 134 passing tests.

> Hosted Meta API, live Meta Llama GPU training, real Hugging Face optimizer resume, Kubeflow and remote MLflow remain unverified.

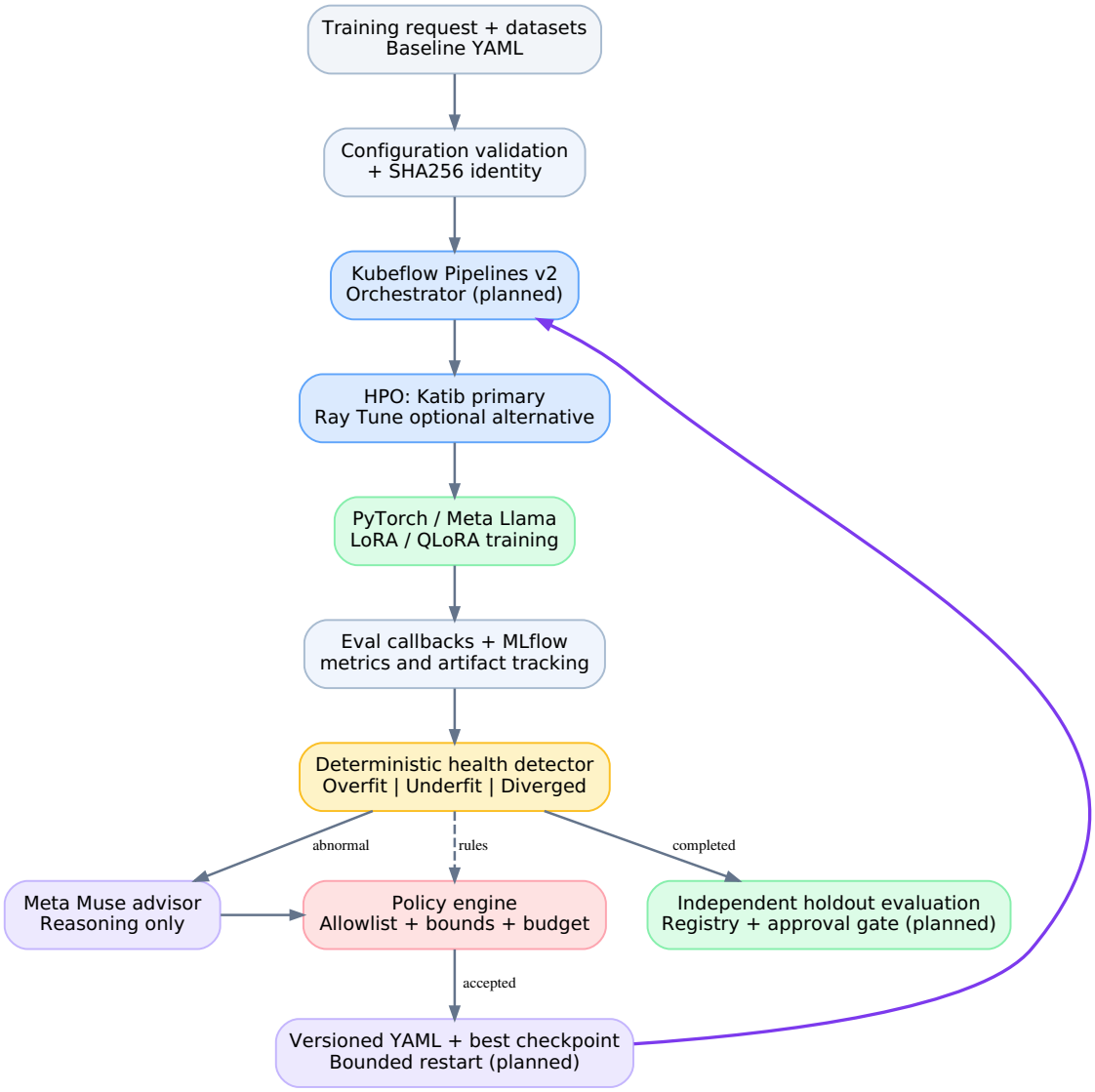
Core outcomes

- Detect likely overfitting, underfitting and non-finite loss during repeated evaluations.
- Capture the best validation checkpoint **from each attempt**, with clear artifact identity.
- Generate small remediation proposals, validate them against strict numeric limits, and rerun only within configured budgets.
- Preserve baseline configurations, decision logs and SHA-256 configuration identity.
- Separate **optimization** (validation metrics) from **final release acceptance** (held-out evaluation, policy, approval).

Read this alongside

- `docs/MASTER_EXECUTION_PLAN.md` - all milestones, interfaces and exit criteria.
- `AGENTS.md` - compact history, decisions, pending items, verified checkpoints.
- `docs/diagrams/*.mmd` - editable Mermaid architecture and logic sources.
- `autotune/` and `tests/` - executable source and test evidence.

1. System architecture



End-to-end architectural components and the feedback loop

Interface explanation. A validated request moves from the configuration layer to an orchestrator. A training job emits evaluation metrics and writes checkpoints. A deterministic health analyzer determines whether to continue or stop. On a detected problem, the Meta agent optionally proposes a bounded patch; the policy engine decides whether to accept it. An approved patch creates a new versioned configuration and a new training attempt. Completion does not imply deployment: the model must pass independent release evaluation.

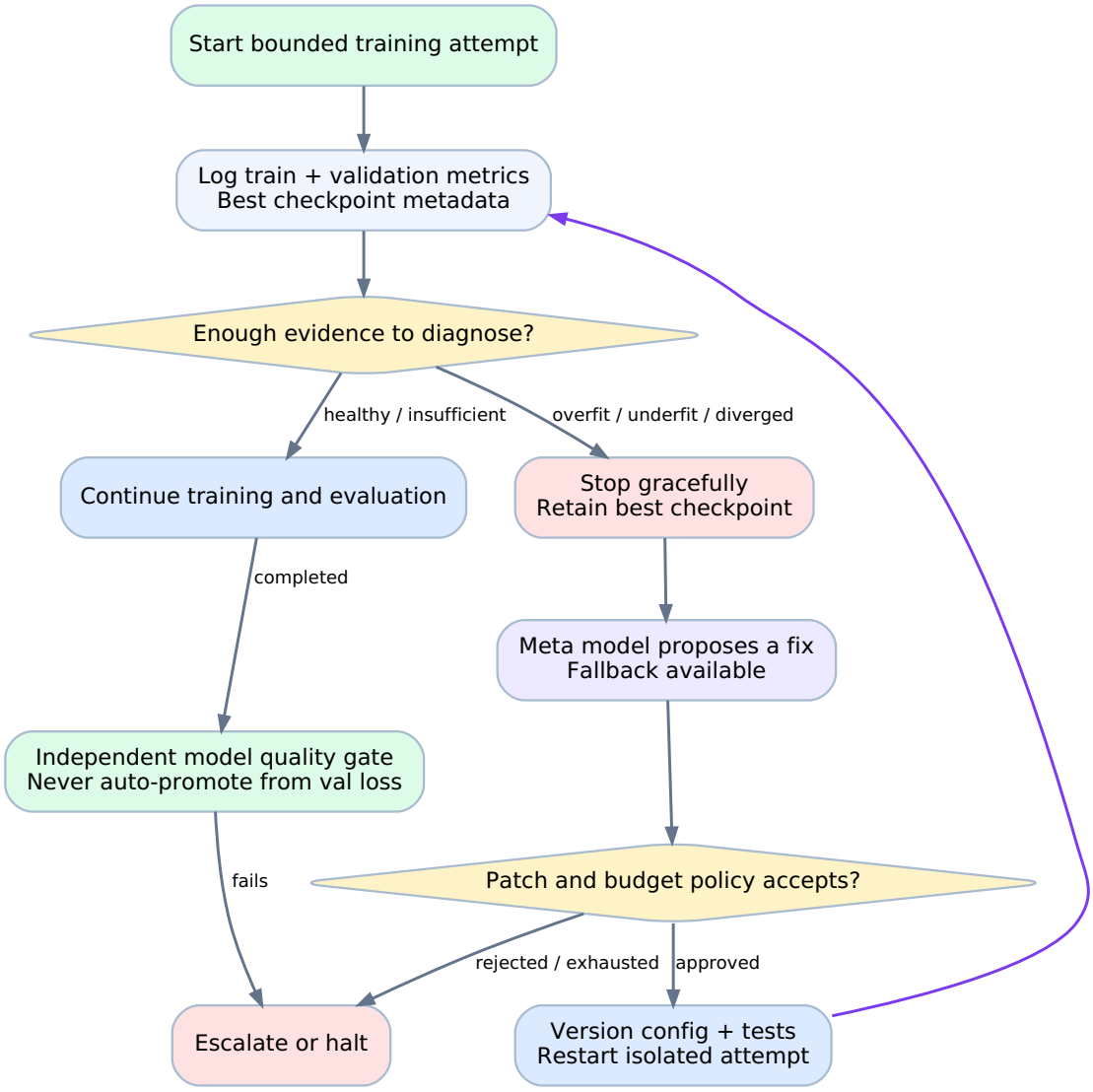
Layer	Primary component	Responsibility	v0.7 status
Control plane	Python CLI / later FastAPI dashboard	Training request, configuration, run controls	Local CLI

Layer	Primary component	Responsibility	v0.7 status
Orchestration	Kubeflow Pipelines v2	Pipeline tasks and retry scheduling	Example only
Optimization	Katib primary; Ray Tune alternative	Controlled hyperparameter search	Standalone Katib sample
Training	PyTorch, Hugging Face, PEFT LoRA	Model adaptation and checkpoints	Real tiny PyTorch CPU weights tested; Meta Llama PEFT GPU unverified
Observability	JSONL, optional MLflow	Metric history, experiments, artifacts	JSONL verified
AI intelligence	Meta Muse Spark advisor	Bounded numeric recommendations and deterministic fallback	Mock end-to-end + mocked transport tests; live API unverified
Trust boundary	Python deterministic policy	Allowlist, ranges, budgets, approval	Patch validation working
Quality	Holdout evaluation + model registry	Promotion assessment	Future phase

Component ownership

Kubeflow orchestrates; Katib optimizes; MLflow records; Meta advises; the policy engine authorizes. Ray Tune may replace Katib for selected experiments, but two independent HPO systems must never own one trial simultaneously. The agent must not read user secrets or raw training records by default.

2. Recovery and stopping lifecycle



Autonomous bounded training and recovery state machine

Controller states

State	Interpretation	Expected action
Insufficient evidence	Early run or missing history	Continue collecting reliable evaluations
Healthy / progressing	No stopping trigger	Continue until planned completion or budget
Suspected overfit	Validation worsens while train loss improves	Preserve best checkpoint; consider more regularization
Suspected underfit	Both losses plateau high relative to task-informed baselines	Diagnose data, capacity, LR and training budget
Diverged	NaN/Inf or unstable optimization	Immediate safe stop; do not blindly resume
Failed	Exception, missing artifact, OOM, policy violation	Audit and escalate or apply a specifically allowed retry
Completed	Attempt finished without stopping trigger	Validate model independently before promotion

Phase 3 now implements a local durable CPU-mock state journal with idempotent receipt reconciliation, explicit blocked states and an operator-run recovery procedure. This is not distributed Kubernetes job cancellation.

Why the best checkpoint matters

Validation loss may be lowest *before* overfitting becomes evident. Save the checkpoint corresponding to the minimum validation loss, rather than just the last epoch. If a hyperparameter change invalidates the optimizer state or adapter structure, begin a new attempt from the base model instead of assuming that warm-starting is safe. A synthetic JSON marker is **not** a restorable weight checkpoint.

3. Training metrics and diagnostic concepts

Training loss measures fit to training examples; **validation loss** helps assess generalization on held-aside validation examples. Their difference is a heuristic signal, not a universal definition of overfitting.

Illustration: overfitting

Train loss trends downward while validation loss first improves and then rises. It can be reasonable to stop only after sustained validation deterioration and a meaningful train/validation gap. Small or noisy datasets can make such heuristics unreliable.

Illustration: underfitting

Both train and validation loss can remain high or flat. Possible causes include too few updates, excessive regularization, weak representation capacity, mismatched labels, bad data, scheduler problems or an insufficient learning rate. A single numerical threshold cannot reliably diagnose underfitting across tasks.

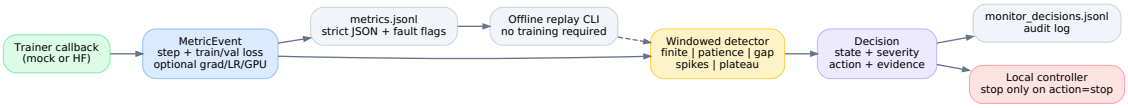
Illustration: divergence

Non-finite loss, gradient overflow or unstable updates merit a safe stop and investigation of learning rate, scaling, data corruption and numeric precision.

Signal	Computation or source	Typical use
Train loss	Trainer callbacks	Optimization progress
Validation loss	Independent validation batches	Checkpoint selection and HPO objective
Generalization gap	`val_loss - train_loss`	Possible overfitting evidence
Validation perplexity	`exp(val_loss)` for token-level cross entropy	Language modeling context only
Accuracy, macro-F1, AUROC	Held-out evaluation tailored to task	Task quality; cannot be inferred from loss alone
LR and gradient norm	Optimizer and trainer hooks	Optimization stability
GPU memory/utilization	Driver and telemetry	Efficiency / OOM prevention
Tokens/sec, GPU hours, spend	Training logs and billing telemetry	Throughput and budget ceilings

Phase 2 now provides: a typed per-evaluation metric contract, warning versus stopping actions, explicit evidence, limited conservative spike/plateau rules, JSONL auditing and offline replay. True step-aligned training windows, generic task metrics and independent GPU probes remain future work.

3A. Phase 2: telemetry pipeline and operational diagnosis



Phase 2 evaluation telemetry, audit logs, decision policy and replay flow

Implementation inventory

Code path	Purpose	Operational status
`autotune/metrics.py` - `MetricEvent`	Typed evaluation sample: positive step, train loss and validation loss; optional timestamp, LR, gradient and GPU peak memory	Tested with synthetic and invalid inputs
`autotune/metrics.py` - `diagnose`	Windowed conservative diagnosis, severity/action classification and numerical evidence	Tested
`autotune/metrics.py` - `MonitorStream`	Accumulate observations, emit a decision for every evaluation to JSONL	Integrated and tested in mock trainer
`autotune/tracking.py`	Strict JSON-safe metric history, nonfinite markers with null values, optional MLflow	JSONL tested; external MLflow not tested
`autotune/monitor_cli.py`	Replay `metrics.jsonl` offline into a comprehensive JSON decision report	Tested
`autotune/mock_trainer.py`	Feed live mock events through monitor and stop only on action=`stop`	End-to-end simulation verified

Code path	Purpose	Operational status
`autotune/hf_trainer.py`	Same monitor call during HF evaluation callback	Source adapted; no real GPU test

Event and decision contracts

```
{
  "step": 6,
  "train_loss": 0.98,
  "val_loss": 1.79,
  "learning_rate": 0.0001,
  "grad_norm": 12.2
}
```

Each event represents the last available train metric paired with an evaluation metric. Required: positive increasing `step`, numeric `train_loss`, numeric `val_loss`. Optional fields: timestamp, learning rate, gradient norm, peak GPU allocation. The logger provides an external UTC audit timestamp. Future work should align train-loss averaging windows precisely to evaluation steps in the HF adapter; the current `on_log` snapshot can lag the validation window.

Example decision structure, stored in `monitor_decisions.jsonl`:

```
{
  "state": "overfit",
  "explanation": "Validation loss rose for 2 evaluations while train loss fell; gap=0.810",
  "severity": "critical",
  "action": "stop",
  "evidence": {"step": 6, "gap": 0.81, "patience": 2, "best_val_loss": 1.57}
}
```

Stop semantics are explicit: the training callback must inspect `action`, not assume every named anomaly means a stop. `info` and `warning` decisions continue training. Severe errors such as nonfinite loss can request termination immediately even before a warmup window completes.

Diagnostic matrix and false-positive protection

Signal	Necessary evidence	Classification	Current behavior
Warmup	Fewer than `min_evaluations` paired events	Info / continue	No stop
Missing or repeated steps	Missing metric or non-monotonic evaluation sequence	Warning / continue	Signals insufficient evidence
Nonfinite value	NaN/Inf loss or gradient norm	Critical / stop	Trial stop; no automatic divergence patch yet

Signal	Necessary evidence	Classification	Current behavior
Isolated train+val spike	Both losses jump above configured ratio in one evaluation	Warning / continue	Require confirmation
Repeated train+val spikes	Multiple consecutive jumps exceed `loss_spike_ratio`	Critical / stop	Stop to avoid uncontrolled divergence
Overfit heuristic	Validation rises every patience observation; train improves; gap exceeds threshold	Critical / stop	Preserve best simulated checkpoint marker
High flat losses	Both losses remain high and near-flat for full window	Critical / stop	Task-specific floor may need calibration
Low flat losses	Both losses flat but not over underfit floor	Warning / continue	Not proof of underfitting
High finite gradient norm	Latest grad norm exceeds `grad_norm_warning`	Warning / continue	Not auto-stop without trend evidence

Underfit and overfit labels are *heuristic findings*; they do not establish that the model is truly underfitting or overfitting across all tasks. A genuine research or production project should calibrate thresholds on data scale, dataset size, model family and training objectives.

Offline replay / reproducibility

From VS Code PowerShell:

```
.\.venv\Scripts\python.exe -m pytest -q
.\.venv\Scripts\python.exe -m autotune.orchestrator --mode mock --no-writeback
.\.venv\Scripts\python.exe -m autotune.monitor_cli --file "runs\YOUR_RUN_FOLDER\trial-01\metrics.jsonl"
        --config config/baseline.yaml --output "runs\monitor_report.json"
```

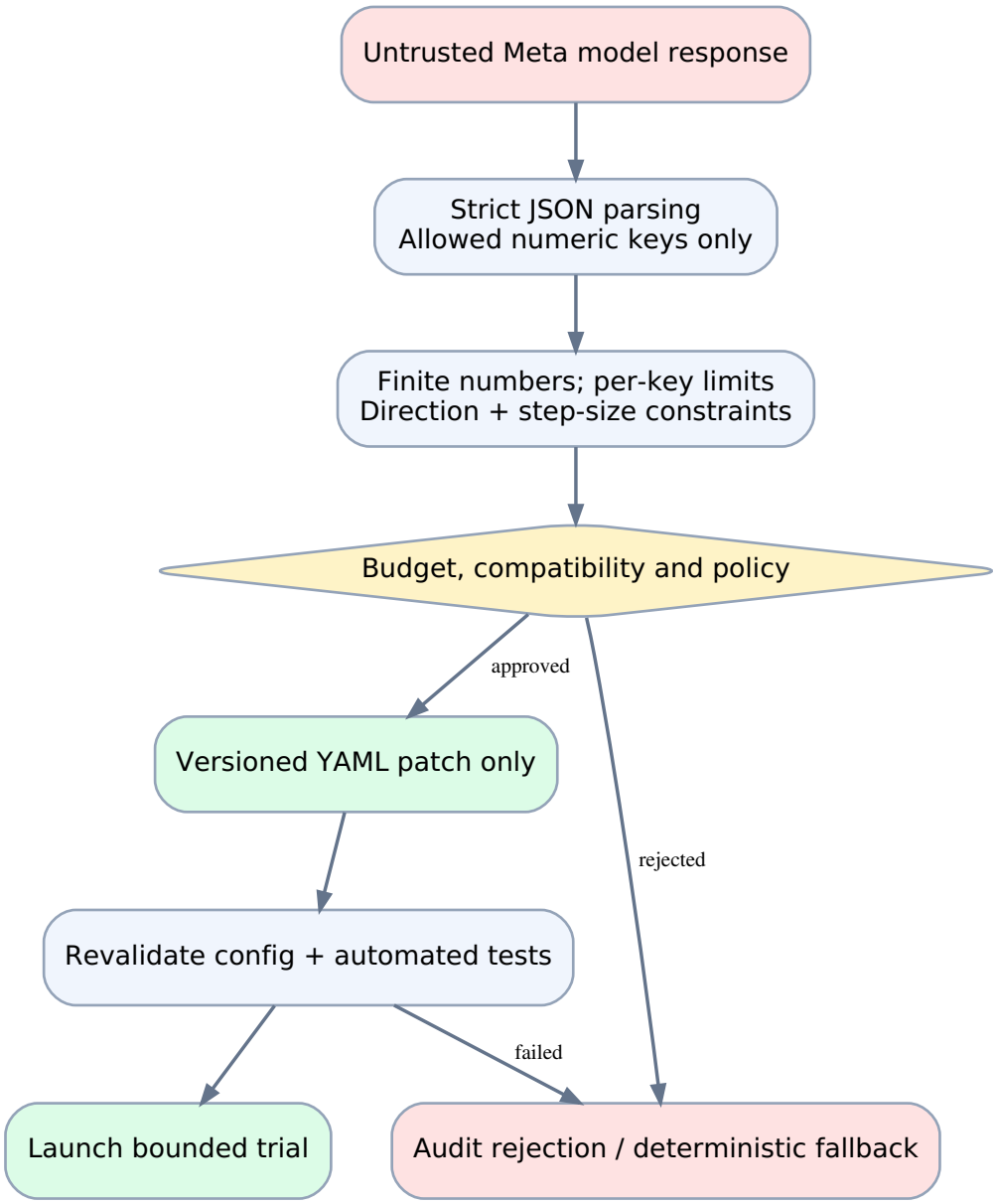
Expected benchmark-free demonstration: 36 local Python tests passed; synthetic overfit at step 6 in attempt one, a second attempt completes, with best **simulated** val loss 0.58. Actual Llama GPU fine-tuning and Kubeflow/MLflow production connections are not evidenced here.

Known technical limitations and next phase

1. `MetricEvent` validates only essential loss fields and limited optional signals; no universal accuracy, F1 or dataset-quality metrics yet.
2. Date/timestamp is recorded but ISO timestamp parsing / clock drift validation is not enforced.
3. Very noisy datasets can defeat monotonic-patience rules; tune thresholds with task-specific baselines before GPU usage.

4. Missing metric records do not stop, though a production controller should escalate after a configurable missing-data timeout.
5. The policy fallback does not auto-remediate divergence. The Phase 3 durable mock controller fails closed instead of retrying blindly.
6. `monitor_decisions.jsonl` is per-trial audit evidence; Phase 3 now adds atomic LOCAL journal and simulated checkpoint validation. Distributed job reconciliation and real checkpoint validation remain future work.

4. LLM safety and codebase change boundary



Trust boundary between model suggestions and executable training decisions

The advisory agent returns a structured object such as:

```
{
  "patch": {
    "learning_rate": 0.00008,
    "lora_dropout": 0.15,
    "weight_decay": 0.03
  }
}
```

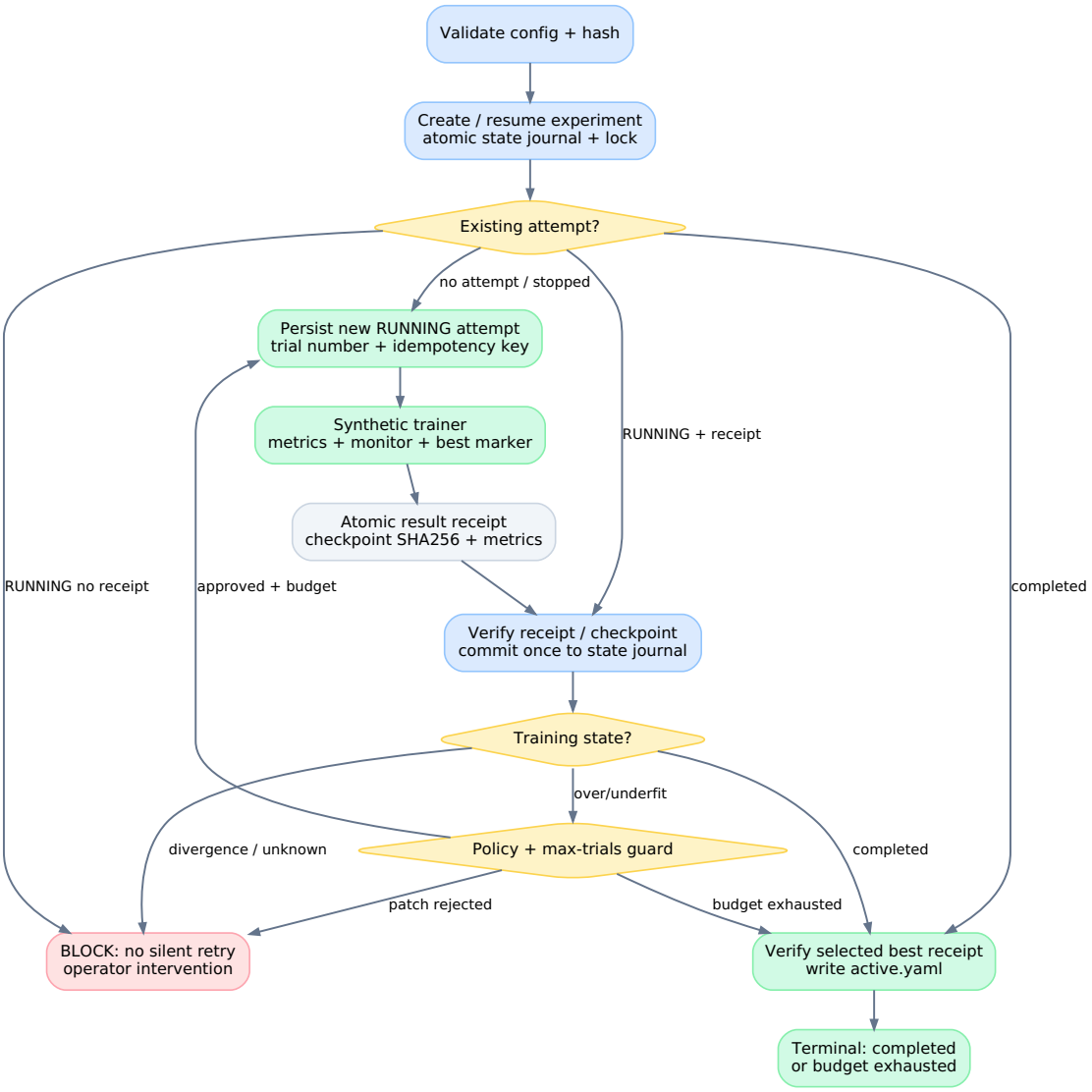
An agent proposal is not permission. The engine checks allowed keys, finite values, absolute bounds, maximum per-attempt change and diagnosis-specific directional rules. Existing v0.1/v0.3 policy supports `learning_rate`, `weight_decay`, `lora_dropout`, `num_train_epochs`, `lora_r`, and `gradient_accumulation_steps`. Any unknown parameter or executable instruction is rejected.

Phase 9 will add **Git branch -> patch -> CI -> review/approval -> merge -> restart** for configuration or limited code changes. We must not let an LLM arbitrarily rewrite `.py`, import untrusted training scripts or run commands. When remote Meta access fails, the deterministic fallback can be used if policy accepts it.

Operational stop conditions

- Retry ceiling reached; no approved patch; no trustworthy checkpoint; invalid/nonfinite metrics.
- GPU or financial budget exceeded; unsupported device precision; repeated OOM or external API failure.
- Unapproved model, unlicensed dataset, data leak risk or failed quality/evaluation gate.

4A. Phase 3: durable and safe recovery (v0.4)



Local journal, checkpoint receipt, idempotent reconciliation and blocked-state control

Why persistence is necessary

In Phase 2 an in-memory process controlled several simulated trials. A process kill could happen after training finished but before an attempt was recorded, resulting in duplicate GPU spending or invalid artifact selection. In Phase 3 each attempted operation has a durable state before starting and an atomic result receipt after finishing. Resume reconciles these two independent records rather than blindly repeating work.

State and transition contract

State or marker	Written when	Resume behavior
`running` attempt	Before invoking the trainer	Inspect `result_receipt.json` first
Valid result receipt	After the trainer returns, with checkpoint SHA256	Commit exactly once, never rerun finished attempt
`stopped`	Verified monitor signal indicates over/underfit or another stop	Only over/underfit with approved patch can retry
`completed` attempt	Finished training without monitored stop	Experiment terminates and checks best checkpoint
`abandoned`	Operator confirmed an unreceipted prior job ended	Trial number counts against budget; retry is explicit
`blocked` experiment	Uncertain job, bad receipt, invalid checkpoint or unsafe condition	Human investigation; no automatic retry
`budget_exhausted` experiment	Trial count reaches max while verified candidate exists	Keep best candidate but no release promotion
`completed` experiment	A complete attempt exists and best marker is verified	No-op on repeated resume

Idempotency: each attempt has a stable SHA256 idempotency key combining experiment ID, trial number and configuration hash. The journal at `recovery_state.json` is the authoritative record; `recovery_events.jsonl` supports incident analysis but can lag a journal transition if the host crashes at the wrong moment. Completion receipts include attempt identity, config SHA256, simulated checkpoint SHA256 and best validation metrics. A correct receipt may be safely committed after restart.

Crash uncertainty: with state=`running` and no valid receipt, the recovery engine will refuse to start another attempt automatically. The operator must verify no training job is alive, then use an explicit abandon action; it consumes a trial slot. Untrusted or mismatched receipts fail closed. A local exclusive `.controller.lock` blocks concurrent writers; clearing a stale lock requires explicit operator acknowledgement that no controller remains active.

Source and executable interfaces

Module / file	Responsibility
`autotune/recovery.py`	Local state machine, atomic JSON, locks, receipts, checksum verification, retry and best-state selection
`autotune/recovery_cli.py`	Windows-friendly new-run and resume CLI; operator interventions require flags
`tests/test_recovery_phase3.py`	Fault injection, idempotent completion, corruption, lock refusal, baseline-change refusal, budget and opt-in writeback
`docs/PHASE3_RUNBOOK.md`	Operator recovery procedure, command examples and safety boundaries
`run_phase3.ps1`	Doctor, test suite and durable synthetic demo

Implementation sequence and restart behavior

1. Validate baseline YAML and calculate its stable SHA256 before creating a fresh experiment.
2. Write a journal, atomically write each attempt config, acquire the experiment lock and record the `running` attempt before invoking the trainer.
3. Receive evaluation metrics from the existing Phase 2 monitor and let the mock trainer create its best-checkpoint **JSON metadata marker**.
4. Hash the artifact and write an atomic `result_receipt.json`; match the trial ID, config hash, best step and loss against checkpoint metadata.
5. Atomically commit the verified result and update `best_config.yaml` only when validation loss improves; if stopped for overfit/underfit, apply a bounded deterministic patch and start another numbered attempt.
6. On completion or exhausted budget, re-verify the winning receipt/marker before writing `active.yaml`. Never run test-set evaluation or automatic model deployment at this point.
7. If interrupted, reconcile a valid receipt without rerunning; block if work is uncertain, corrupted or unsupported. Resume requires the same baseline config fingerprint.

```
# Phase 3 new CPU-mock experiment
.\.venv\Scripts\python.exe -m autotune.recovery_cli --config config/baseline.yaml --workspace runs

# Reconcile a previous experiment after verifying no active external job
.\.venv\Scripts\python.exe -m autotune.recovery_cli --config config/baseline.yaml --resume
        'runs\YOUR_EXPERIMENT'

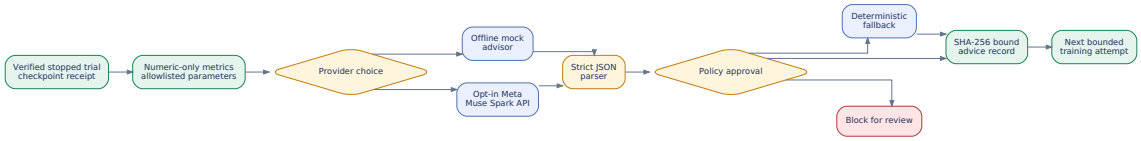
# Only if prior job definitely stopped and no receipt exists
.\.venv\Scripts\python.exe -m autotune.recovery_cli --config config/baseline.yaml --resume
        'runs\YOUR_EXPERIMENT' --abandon-interrupted
```

Validated tests and limitations

Verification: 48/48 tests passed locally on Python 3.13, adding 12 Phase 3 tests to the 36-test v0.3 suite. Simulated failure points include after recording RUNNING, after training but before receipt, and after receipt but before journal commit. Valid receipts reconcile without duplicated metrics. Corrupted checkpoint hashes prevent selection; the abandoned trial counts against the retry ceiling. A second identical resume on a terminal experiment does not emit new events.

Limits: Phase 3 only supports synthetic CPU training through the new durable CLI. It does not yet implement durable PyTorch weight checkpoints, cross-process Kubernetes job reconciliation, distributed locks, real hardware cost quotas, codebase Git commits or live Meta API reasoning. `SHA256` proves local bytes match a recorded digest, not that the model is trustworthy or loadable. The old `autotune.orchestrator` is retained for backward compatibility, but it is not the new crash-safe controller. Testing on the user's Windows host is still required.

4B. Phase 4: Meta advisor integrated into durable recovery (v0.5)



Strict advisory trust boundary and retry integration

What was actually implemented

Phase 4 introduces `autotune/advisor.py` and `autotune/advisor_cli.py`, connects validated advice to Phase 3's `run_durable` controller, and adds `config/agent_mock.yaml` for zero-cost CPU validation. The same recovery loop can be configured for a hosted Meta Muse Spark recommendation, but **a successful real external API call has not been executed or claimed**. The default `config/baseline.yaml` continues using the pre-existing deterministic rule engine, protecting Phase 3 compatibility.

The Meta endpoint and model identifiers are drawn from Meta's public Model API documentation (checked 2026-11-01): `https://api.meta.ai/v1` and `muse-spark-1.3`, through an OpenAI-compatible Chat Completions request. An API key remains in the `MODEL_API_KEY` environment variable, never in the training configuration. The optional network call is bounded by a 20-second client timeout and zero client retries. Offline mock and network failure do not require credentials.

Interface contracts

Contract	Input	Valid output	Rejection / fallback
Advisor input	Diagnosis + last eight allowlisted metric dictionaries + six numeric training parameters	Sanitized JSON only	Raw samples, commands, file paths and secrets never included
Model response	Strict JSON object with `patch`, optional matching `diagnosis` and short `rationale`	Numeric patch with allowlisted keys	Non-JSON, duplicate JSON keys, unknown fields, oversized output, contradictory diagnosis and bad types rejected
Policy engine	Proposed patch + current configuration + original monitor diagnosis	Bounded direction-aware hyperparameter delta	Unapproved values and changes rejected, deterministic safe fallback attempted
Durable snapshot	Trial number, config SHA-256, checkpoint SHA-256, validated patch, output config SHA-256	`trial-XX/advisor_decision.json` with its own hash	Altered or inconsistent saved decision blocks replay
Controller	Validated candidate configuration, finite attempt budget	New versioned CPU-mock training attempt	Divergence or unsupported cause blocks automation

Why this boundary matters

A language model can output malformed JSON, non-finite numbers, duplicate keys, off-policy parameter changes, or text designed to trigger dangerous actions. Phase 4 handles all model replies as **untrusted data**: the assistant cannot request terminal access, execute arbitrary code, see training records or secrets, edit Python files, or authorize promotion. A separate Python validator checks every suggested parameter. The saved recommendation is bound to the exact stopped attempt and checkpoint and is **replayed rather than recomputed** after an interruption.

The deterministic fallback does not promise better performance: it only makes a bounded, policy-consistent candidate. The model must still be trained and evaluated before any quality conclusion. A CPU mock scenario uses synthetic losses and a JSON metadata checkpoint, not real gradients or Meta Llama weights.

Example synthetic recovery trace

1. Trial 1 emits falling train loss but rising validation loss after its optimum; the monitor stops it with an overfit diagnosis.
2. Phase 3 validates the mock checkpoint marker digest and commits the receipt to durable state.

3. The Phase 4 mock advisor proposes `lora_dropout: 0.15`, `weight_decay: 0.03`, and `learning_rate: 0.00008` from starting values `0.05`, `0.01` and `0.00010`.
4. The deterministic policy verifies values and directions. The controller writes `trial-01/advisor_decision.json` with both config hashes and the checkpoint hash.
5. The controller starts trial 2 from the baseline simulator with the new configuration. The scripted loss curve ends at validation loss **0.58**, which is **synthetic** and not a real model accuracy claim.
6. On restart, a saved advisor decision is independently revalidated; a tampered snapshot blocks instead of making a fresh API call and overwriting audit history.

CLI implementation and reproducibility

```
py -3.13 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m pytest -q
.\.venv\Scripts\python.exe -m autotune.advisor_cli --provider mock --diagnosis overfit
.\.venv\Scripts\python.exe -m autotune.recovery_cli --config config/agent_mock.yaml --workspace runs
```

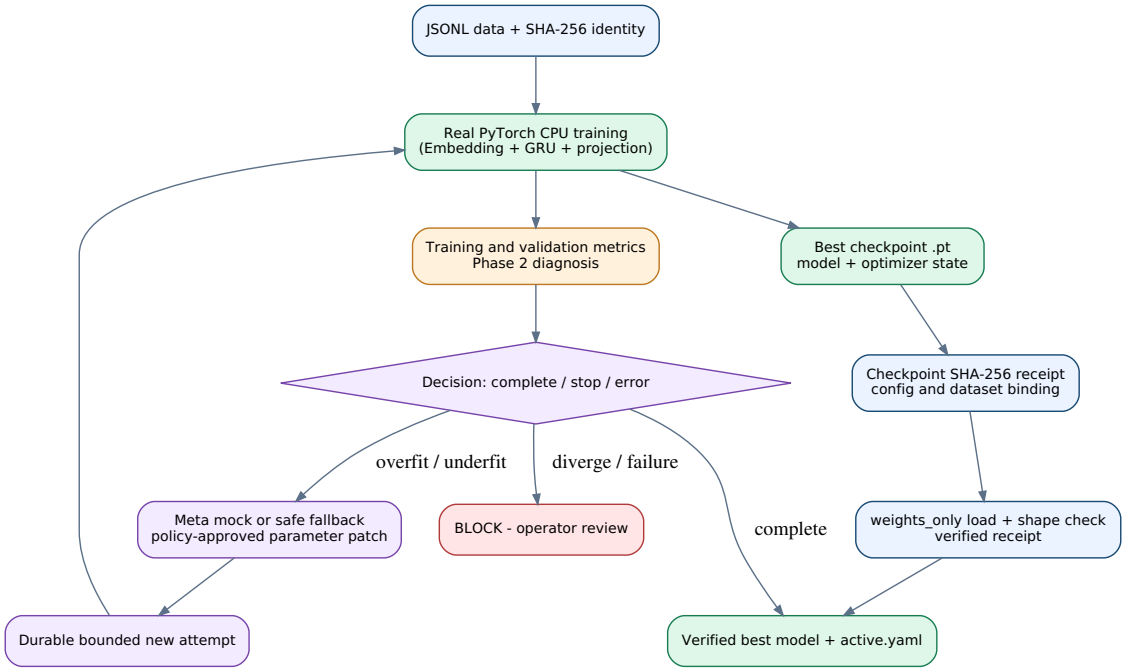
An optional hosted test is **explicit**: set `MODEL_API_KEY` securely and invoke `python -m autotune.advisor_cli --provider meta --diagnosis overfit` (after installing the optional `openai` client). API access might be billable; absence of a key produces a visible deterministic fallback, not fabricated remote inference. See `docs/PHASE4_RUNBOOK.md` for Windows and PowerShell specifics.

Verification evidence: 77/77 local Python tests passed, including 29 new Phase 4 tests for malformed/adversarial responses, injection-safe payload selection, missing credentials, replay consistency, tamper rejection and an end-to-end synthetic durable recovery. The actual hosted endpoint was not contacted. No GPU, Kubeflow cluster, actual weight checkpoints, or real measured improvements were used.

4C. Phase 5A - Genuine PyTorch training with verified weight checkpoints (v0.6)

> **Verification scope:** A small byte-level GRU language model was genuinely trained on the bundled JSONL instructions using offline PyTorch CPU, with real backpropagation and optimizer updates. The optional Hugging Face Meta Llama LoRA adapter in `autotune/`

> `hf_trainer.py` is still NOT GPU-tested; this is not a claim of real Meta Llama fine-tuning or state-of-the-art quality.



Real weight training, auditing, and bounded recovery

Why this phase is different from the earlier simulation

Phases 1-4 replayed deterministic simulated metrics and stored `best_checkpoint.json` as **metadata**, not model weights. Phase 5A introduces `autotune/real_training.py`, which computes token-level cross-entropy, calls `loss.backward()`, updates parameters with AdamW, evaluates on a separate validation set and saves actual tensors to `best_checkpoint.pt`. The byte model uses an embedding, a GRU and a linear vocabulary projection; it is randomly initialized and is not a pretrained Meta model. Phase 5A does **not** claim fine-tuning of pretrained weights.

End-to-end implementation

1. **Dataset access and provenance.** Read JSONL records with a mandatory `text` field. Require at least two records per split, separate training and validation content, and a bounded local file size. Calculate a SHA-256 identity over both file digests and their explicit split roles. Do not send text records to the advisor.
2. **Tokenization and labels.** Encode UTF-8 into byte IDs (0-255). Input tokens predict their next byte. Mask padding positions with

`-100` in cross-entropy. This is an intentionally lightweight architecture for testing rather than a production text tokenizer.

3. **Weight updates.** Train with AdamW, gradient clipping and deterministic example order. Each epoch produces an average training loss and a separately computed validation loss. The training loss is averaged during the epoch, not recomputed after the epoch; it is a trend estimate rather than perfectly synchronized with validation loss.
4. **Signal monitoring.** Write metrics to `metrics.jsonl`, diagnostic decisions to `monitor_decisions.jsonl`, and run the existing evidence-window rules to stop at an epoch boundary when warranted. These heuristics may misclassify short or noisy runs.
5. **Checkpoint save.** On each improvement, atomically replace `best_checkpoint.pt` with model weights, optimizer state, epoch, validation score, configuration hash and dataset hash. This supports verification and loading; partial-epoch optimizer resume is not yet implemented.
6. **Durable controller.** `autotune/real_recovery.py` uses a separate `real-torch-v0.6` journal schema, creates a receipt containing the exact file SHA-256 and selects a best checkpoint only after verifying checksum, identities, state dictionary shapes and optimizer loadability with `torch.load(weights_only=True)`.
7. **Controlled recovery.** A stopped overfit/underfit attempt can request an allowlisted patch through the existing offline advisor/fallback engine and start a new bounded attempt. The model restarts from a fresh initialization when parameters change. Uncertain RUNNING work without a receipt is blocked; no silent re-execution occurs. Divergence also blocks rather than blindly retrying.

Actual measured CPU smoke test

Epoch	Train loss	Validation loss
1	5.5887	5.6072
2	5.5706	5.5941
3	5.5538	5.5808
4	5.5384	5.5675

The smoke test completed one trial and selected epoch 4; its actual tensor checkpoint was hash-verified and loadable. These values demonstrate execution only, not model quality or comparison with an established baseline. A deliberate threshold-manipulation test also

exercises two real-weight attempts and budget exhaustion; its *underfitting classification is forced for state-machine coverage*, not proof of a genuine learning failure.

Verification evidence: 95/95 automated tests passed locally with PyTorch CPU (77 inherited plus 18 new tests), including real gradient backpropagation, JSONL/dataset integrity, weight and optimizer checkpoint loading, digest/tamper failures, completed-run idempotency, dataset mutation rejection, uncertain RUNNING blocks and bounded retry. The user subsequently confirmed Phase 5A on Windows: 95 tests passed and a loadable tensor checkpoint was produced.

Run on Windows in VS Code

```
py -3.13 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements-real-cpu.txt
.\.venv\Scripts\python.exe -m pytest -q
.\.venv\Scripts\python.exe -m autotune.real_cli --config config/real_cpu.yaml --workspace runs_real
```

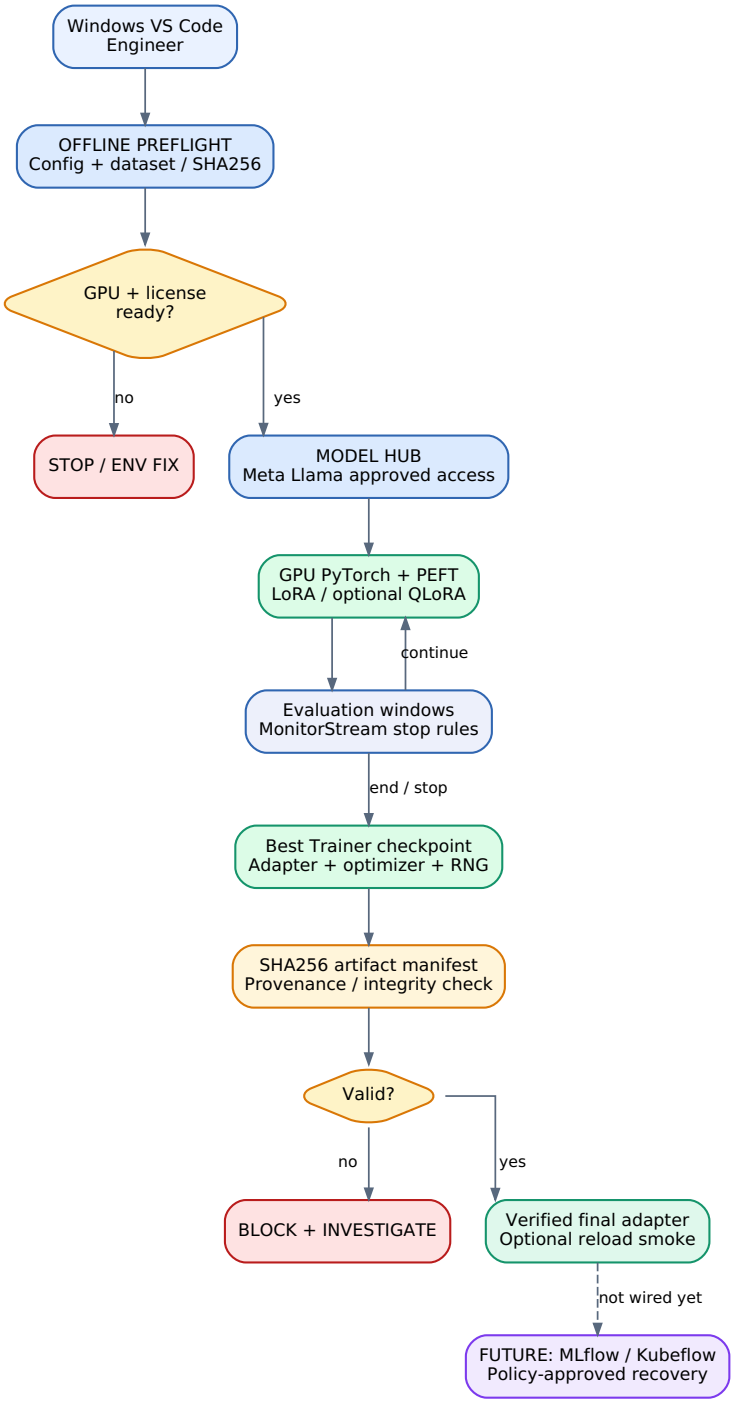
If script execution is allowed, `._phase5.ps1` runs the tests and CPU training. Do not set system-wide unrestricted PowerShell execution policies merely to run it. Inspect `runs_real/<id>/trial-01/best_checkpoint.pt`, `result_receipt.json`, `recovery_state.json`, `active.yaml`, and event/metric logs. The phase requires PyTorch and may need a compatible wheel for Windows/Python; no API key, GPU or network model download is necessary after installation.

Stage 5B background and outstanding live Meta Llama validation gate

The existing `autotune/hf_trainer.py` is **source only and not GPU validated**. Before claiming full Phase 5, exercise a licensed Meta Llama variant with the supported Transformers, PEFT, Accelerate and GPU stack, validate tokenizer and masking semantics, use aligned metrics, verify adapter reload/inference, save optimizer/scheduler state where supported, handle interrupted trials and record GPU memory and data/model revision hashes. The CPU GRU checkpoint is **not interchangeable** with HF/LoRA checkpoints. Neither pipeline is production release-ready.

4D. Phase 5B - Meta Llama / Hugging Face LoRA integration (v0.7)

> **Verification boundary:** The new LoRA pipeline is source-complete for an optional, explicitly triggered pilot and passed offline contract tests. No Meta model weights were downloaded, no GPU fine-tuning was run, and Hugging Face Trainer’s real checkpoint behavior has not been exercised in this build. The user’s Phase 5A Windows run (95 tests) trained the separate tiny-GRU model, not Llama. This release advances integration readiness, not a completed on-GPU milestone.



Guarded Meta Llama LoRA pathway and future MLflow/Kubeflow bridge

Why LoRA and QLoRA are useful

LoRA (Low-Rank Adaptation) freezes a pretrained base model and trains a small number of low-rank update matrices in selected layers. This reduces the number of trainable parameters compared with full fine-tuning. QLoRA loads base weights at reduced precision (commonly

4-bit) while learning LoRA adapters. Neither approach guarantees lower loss or better model quality; memory efficiency and stability depend on the actual hardware, libraries and dataset.

On a GPU server, the plan selects a Meta Llama model, validates authorized model access, tokenizes train/validation JSONL, attaches PEFT adapters to `q_proj` and `v_proj`, and runs Hugging Face Trainer. The offline plan does not make external requests and rejects missing/duplicate splits and configurations that schedule no validation. A separate public `sshleifer/tiny-gpt2` CPU config provides an optional online dependency smoke test.

New code and ownership

Module	Responsibility	Verified scope
`autotune/lora_contract.py`	Offline config/data hash check, split duplication detection, evaluation schedule plan, SHA256 checkpoint manifests	39 new offline tests together with CLI/guardrails
`autotune/lora_cli.py`	Default safe preflight; explicit `--execute`, `--resume-checkpoint`, `--verify-adapter`	Offline planning tested
`autotune/lora_gpu.py`	PEFT LoRA/optional QLoRA, GPU guard, HF Trainer callbacks, adapter save/reload	Source-level integration; real HF run not yet tested
`config/lora_meta_gpu.yaml`	Licensed Meta Llama 3.2 1B pilot config	Validated, not downloaded
`config/lora_public_cpu.yaml`	Public tiny GPT-2-derived CPU training example	Validated, not downloaded
`docs/PHASE5B_RUNBOOK.md`	Windows and optional remote AMD ROCm/NVIDIA GPU steps	Documented, not executed

Correct metrics and checkpoint semantics

The Hugging Face callback collects a training-loss log at the same evaluation cadence (currently `logging_steps == eval_steps`). At evaluation, it pairs the mean of newly logged training-loss values with the validation loss. If no corresponding training loss exists, it writes an `unpaired_evaluation` event and **does not invent a pair or issue a stop**. The monitor uses existing conservative heuristics and can request the Trainer to stop and save at the next safe control boundary.

A *full training checkpoint* includes an adapter `adapter_model.safetensors`, `adapter_config.json`, `trainer_state.json`, `optimizer.pt`, `scheduler.pt`, and RNG state. The separate `final_adapter/` is suitable for inference reload but **is not guaranteed to restore training optimizer state**. Hash manifests bind artifact files to

configuration/model/dataset identities and reject missing, extra or modified files. Hashes are not signatures; untrusted optimizer pickle files must never be resumed.

Important integration boundary: The old Phase 5A tiny-GRU `.pt` checkpoint and the new HF Trainer/PEFT checkpoint are not interchangeable. The HF pilot is not yet attached to Phase 3/4 durable self-healing retry logic. An operator must explicitly start a GPU run and explicitly select a trusted in-run checkpoint to resume. Automatic GPU parameter tuning and requeue are future work.

Agent boundary, model licensing and compute

Meta Muse Spark remains a *diagnostic advisor* in the existing CPU controller, never a shell executor or direct repository editor. It is **not automatically called** by the new HF pilot. Meta Llama model downloads require approved license access through Hugging Face. The base model revision is configurable; default `main` is a moving reference and must be pinned to a commit for strict reproducibility. Trusting remote model code is disabled in the pilot.

For AMD MI300X, select compatible Linux ROCm PyTorch wheels and validate the relevant PEFT and bitsandbytes build before enabling 4-bit QLoRA. `torch.cuda.is_available()` is also used by ROCm-enabled PyTorch; a Windows CPU installation alone cannot satisfy GPU deployment. For NVIDIA use a compatible CUDA stack. No GPU throughput, GPU memory or cost results are claimed in this edition.

Phase 5B test record and Windows validation

The user independently verified **95 passed in 34.36s** on their Windows host in Phase 5A, plus a real tiny-GRU best validation loss `5.567494869232178` and SHA256 checkpoint receipt. A PyTorch NumPy warning was nonfatal, so the new `requirements-real-cpu.txt` includes `numpy`. This v0.7 build adds 39 offline tests, bringing the build environment to **134/134 passing**. Tests cover invalid LoRA configurations, missing validation windows, data leakage, tampered/missing artifact files, hash-bound provenance, symlinks, locking, GPU absence and token label padding. **Fake test artifact bytes do not constitute trained adapter weights.**

On Windows in VS Code (no HF model download):

```
.\.venv\Scripts\python.exe -m pip install numpy
.\.venv\Scripts\python.exe -m pytest -q
.\.venv\Scripts\python.exe -m autotune.lora_cli --config config/lora_meta_gpu.yaml
```

Optional public online CPU smoke and GPU execution commands are detailed in `docs/PHASE5B_RUNBOOK.md`. Execute those before marking the actual GPU/adapter gate complete. This is **not a production benchmark or safety certification**.

Risks and next release gate

The live HF/PEFT dependency matrix, Meta repository permissions, compatibility of checkpoint file layouts across Trainer versions, ROCm/CUDA kernels, actual optimizer resume, model inference, GPU OOM and VRAM metrics remain to be validated. The training callback is conservative and does not prove that two correlated loss numbers represent equivalent distributions. Final quality evaluation must use an untouched holdout and a separately approved promotion policy.

Next milestone: Phase 6 after a verified LoRA GPU run - connect real adapter artifacts and evaluation metrics to a persistent MLflow tracking server; only then integrate those jobs with durable Kubeflow-owned recovery. Continue to refresh the PDF handbook, roadmap and AGENTS.md for every active iteration.

5. Full phased implementation program

Phase	Scope / deliverable	Exit gate	State
0	Requirements, architecture, docs	Baseline agreed, living docs present	Complete
1	Package, validation, doctor, tests	Local mock and tests pass	Complete
2	Metric contract, diagnostics	Noisy/missing/divergent cases tested	Complete - 36 total tests at Phase 2
3	Recovery state machine	Idempotent stop/ checkpoint/retry tests	**Complete - 48 total tests**
4	Meta diagnostic agent	Strict JSON and adversarial-output tests	Complete - 77 total CPU-mock tests at Phase 4
5	Real PyTorch CPU (5A) + Llama LoRA (5B)	Loadable CPU weights; then supported GPU adapter validation	**5A user-verified; 5B contract tested (134 total); live GPU pending**
6	MLflow infrastructure	End-to-end run lineage and artifact store	Planned
7	Kubeflow deployments	Cluster pipeline launches bounded attempts	Planned
8	Katib / Ray Tune	Verified optimizer selection and trial search	Planned

Phase	Scope / deliverable	Exit gate	State
9	Git-reviewed changes	PR, tests, rollback, approval policy	Planned
10	Unified dashboard	Live metrics, timeline, controls and alerts	Planned
11	Quality/policy gates	Held-out evaluation and model registry	Planned
12	Production hardening	Reproducible installation and tested runbook	Planned

Critical path: Phases 0-4 completed as CPU mock; Phase 5A real tiny PyTorch weights verified. **Next:** Phase 5B live GPU verification, then Phase 6 MLflow. Add MLflow persistence before moving recovery onto independent Kubernetes jobs; then add Katib. The dashboard can grow incrementally but the final promotion gate follows robust evaluation.

Deliverable acceptance by phase

Phase 2: typed metric events, paired train/eval event contract, robust anomaly tests, diagnosis evidence.

Phase 3 (mock complete): durable local transitions, attempt keys, simulated checkpoint SHA256 verification, failure injection, safe resume. Real weight-checkpoint compatibility remains Phase 5+ work.

Phase 4 (CPU mock complete): strict Meta response schema, offline integration, privacy allowlists, failed-call fallback, persisted advisor decision and tamper/replay tests; hosted API connection still unverified.

Phase 5A: real small PyTorch CPU weights + checkpoint validation verified. **Stage 5B outstanding:** actual Meta Llama LoRA adapter weights and repeatable validation loss from a supported GPU.

Phases 6-8: MLflow tracking, Kubeflow job isolation and Katib-controlled search with safe ownership.

Phases 9-12: governed Git changes, UI, holdout and model registry, deployment/performance/security checks.

6. Phase 1 - implementation record

Starting point: Existing v0.1 CPU-only synthetic controller, metric heuristics, patch allowlist, optional Meta/MLflow hooks, HF LoRA adapter, Kubeflow pipeline definition and Katib example. The user-provided starter's historical tests were 8 passing; no real GPU/KFP deployment had been verified.

Added in v0.3:

1. `autotune/config.py` validates baseline structure and critical parameters before a run starts. It rejects invalid numeric values, booleans masquerading as numeric values, malformed YAML, invalid budgets, invalid LoRA ranks and identical train/validation files. It produces a content-stable SHA-256 config fingerprint.
2. `autotune/doctor.py` exposes `--mode mock|hf` to check the local environment and emit JSON status without printing API secrets.
3. `autotune/orchestrator.py` now loads the validated configuration before creating a run directory and puts the SHA-256 content identity into the experiment manifest.
4. `tests/test_config.py` adds seven tests across contracts, invalid values, no-side-effect failures and environment checks.
5. Master plan, diagrams and maintained AGENTS.md have become first-class project artifacts.

Test results from this session

```
python -m pytest -q
...............     [100%]
15 passed in 0.12s

python -m autotune.doctor --mode mock
"ok": true
```

Synthetic recovery: attempt 1 detected overfitting at evaluation step 6; its best simulated validation loss was 1.57 at step 4. Attempt 2 completed the mock curve with best validation loss 0.58. These numbers are intentionally generated by a deterministic test harness and **must not be cited as real model accuracy or comparative improvements.**

Environment of verification: Python 3.13 in the working container, with installed PyYAML and pytest. Commands for Windows Python 3.11 are provided but have not been executed on the user's machine.

7. How to run Phase 1 in VS Code (Windows)

Extract the project ZIP, open `AutoTune_AI_Phase1` in VS Code and use its integrated PowerShell terminal. From the extracted project root:

```
py -3.11 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m autotune.doctor --mode mock
.\.venv\Scripts\python.exe -m pytest -q
.\.venv\Scripts\python.exe -m autotune.orchestrator --mode mock --no-writeback
```

The final command creates a new `runs/<timestamp>/` experiment directory with `manifest.json`, `summary.json`, event history, trial metrics, YAML configs and synthetic checkpoint metadata. `--no-writeback` leaves `config/optimized.yaml` unchanged; remove that flag when explicitly testing controlled config writeback.

For optional MLflow + dashboard, install `requirements-optional.txt` and configure a local tracking server as outlined in the README. This is optional for the local simulation; it is not necessary for the 48-test suite.

Common problems and resolutions

Symptom	Likely cause	Next action
`py -3.11` not recognized	Python launcher/Python 3.11 not installed	Install Python 3.11 and verify `py --list`
PowerShell activation blocked	Execution-policy restrictions	Call `.venv\Scripts\python.exe` directly; no activation required
`ConfigError` on baseline	Wrong key/type/bounds	Read error; correct YAML, rerun doctor
Meta advisor unavailable	No API key/network/provider access	Use deterministic fallback; no secret in Git
No evaluation in HF run	Too few training steps vs `eval_steps`	Increase dataset/epochs or reduce eval interval
Missing model checkpoint	Synthetic `best_checkpoint.json` is metadata	Real GPU adapter must be trained before restoration
Kubeflow example cannot run	No compatible Kubernetes/KFP infrastructure	Complete Phase 7 cluster preflight

8. Proposed deployment and GPU strategy

Development: Windows + VS Code, CPU-only simulator, unit tests, local files, Git branches. The first real GPU milestone can run on a supported NVIDIA CUDA environment or a validated AMD ROCm environment, depending on model precision and library compatibility. Never assume a package built for one compute stack works on another without checking drivers, PyTorch and PEFT versions.

Production target: Container registry + Kubernetes namespace, GPU device plugin, Kubeflow Pipelines, Katib, MLflow tracking service and external durable artifact storage. Put dataset/model credentials in Kubernetes Secrets with least privilege. Keep run IDs, dataset revisions, model revisions and auditable policy decisions connected. Introduce GPU-hour budgets, storage limits, cancel paths and approval before auto-deployment.

Data and model governance

Training data must be licensed and separated into train/validation/final holdout partitions. Record dataset fingerprints, task definitions and model revision identities. A final holdout set is evaluated sparingly and cannot feed the agent’s optimization loop. Use model cards and evaluation reports before registry promotion. Phase 5A implements separate train/validation fingerprints; final untouched holdout and production model revision governance remain outstanding.

9. Next task and checkpoint protocol

Next verification gate is running Stage 5B on a supported GPU, then connecting Phase 6 MLflow. Replace synthetic checkpoint markers with loadable model/adapter weights, enforce correct training/evaluation step alignment, save compatible optimizer state when needed, and connect the existing Phase 4 advisor to measured training curves. Live Meta endpoint testing remains an optional separately verified gate.

At the end of every future AutoTune AI project iteration:

1. Run the relevant tests and report true pass/fail evidence.

2. Update `docs/MASTER_EXECUTION_PLAN.md` and the current phase status.
3. Update this handbook source with new diagrams/concepts/implementation and generate the next PDF edition.
4. Append a dated checkpoint, architecture decisions, errors, next steps to `AGENTS.md`.
5. Distribute a new project ZIP and the PDF. Never silently claim an integration was live-tested.

Sources and references

- Kubeflow Pipelines v2 compiling: https://www.kubeflow.org/docs/components/pipelines/user-guides/core-functions/compile-a-pipeline/
- Kubeflow Katib overview: https://www.kubeflow.org/docs/components/katib/overview/
- Katib early stopping design: https://www.kubeflow.org/docs/components/katib/user-guides/early-stopping/
- MLflow experiment tracking: https://mlflow.org/docs/latest/ml/tracking

Phase 5B CPU-first runbook - AutoTune AI v0.7.2

Decision (2026-11-01): User has no dedicated GPU. CPU is the default execution target; GPU stages are optional. The existing Phase 5A GRU is genuinely Windows-verified (95 tests). Phase 5B Hugging Face LoRA/PEFT is an unverified online smoke path until run. Meta Muse diagnostic API can be called from CPU; remote service performs inference. Do not claim that hosted Meta calls have been tested.

What this changes

- Keep CPU training, diagnostics, checkpoint/recovery contracts and policy engine as primary deliverables.
- Use `config/lora_public_cpu.yaml` (small public `sshleifer/tiny-gpt2`, `hf.device: cpu`, `quantization: none`), not `config/lora_meta_gpu.yaml`.
- No 4-bit QLoRA on CPU in this pilot. Do not try licensed Meta Llama on local CPU using this configuration.

- Kubeflow/Katib, MLflow and the dashboard can be developed on CPU, although Kubernetes resource overhead should be measured. GPU benchmarking is optional and postponed.
- Model-card and validation metrics from a tiny test model only demonstrate pipeline correctness, not model quality or scaling.

Windows PowerShell commands (from extracted project root)

First, with existing venv for offline tests:

```
.\.venv\Scripts\python.exe -m pytest -q
.\.venv\Scripts\python.exe -m autotune.lora_cli --config config/lora_public_cpu.yaml
```

For genuine HF PEFT CPU LoRA, create a fresh Python 3.11 virtual environment so working Phase 5A packages remain intact:

```
py -3.11 -m venv .venv-hf
.\.venv-hf\Scripts\python.exe -m pip install --upgrade pip
.\.venv-hf\Scripts\python.exe -m pip install torch
.\.venv-hf\Scripts\python.exe -m pip install -r requirements-lora.txt
.\.venv-hf\Scripts\python.exe -m autotune.lora_cli --config config/lora_public_cpu.yaml
.\.venv-hf\Scripts\python.exe -m autotune.lora_cli --config config/lora_public_cpu.yaml --execute
        --allow-cpu --output outputs/public-cpu-01
```

This requires internet access once to download the public model and dependencies, and CPU training could be slow. Do not reuse `outputs/public-cpu-01` when it already has output; choose another run directory.

After real training **if it completes successfully**, verify the LoRA adapter can reload:

```
.\.venv-hf\Scripts\python.exe -m autotune.lora_cli --config config/lora_public_cpu.yaml
        --verify-adapter outputs/public-cpu-01/final_adapter --allow-cpu
```

Inspect `outputs/public-cpu-01/metrics.jsonl`, `monitor_decisions.jsonl`, `result_receipt.json`, `checkpoints/`, and `final_adapter/`. Full resume from a Trainer checkpoint and separate autonomous retry are distinct features and require their own live tests.

Verification and next milestones

1. Existing baseline: 95/95 user Windows verified for Phase 5A; 134/134 build-env verified for v0.7 Phase 5B offline tests. v0.7.2 adds a CPU routing/CLI test; package tests should be rerun.

2. CPU PEFT first live gate: actual adapter train, nonempty validation metrics, checksum manifest, adapter reloaded, safe operator review.
3. CPU checkpoint/resume gate: interrupt a run at a safe point, inspect full trainer state/manifest, restart and compare reproducible state.
4. CPU MLflow -> agent proposal -> deterministic guard -> second trial -> best validation comparison -> audit log. This is the real self-healing integration target without GPU.
5. CPU Kubernetes/Kubeflow/Katib optional after standalone loop verified; GPU is not a blocker.

Known limitations

- Live Hugging Face/PEFT training has NOT been tested during this documentation/CPU-route update. Versions of transformers, peft and PyTorch may require compatibility adjustments.
- Monitoring of real HF training requires matched train/eval metrics; current pilot records averaged training windows.
- `final_adapter` is not a complete optimizer/scheduler state; only a verified Trainer checkpoint may resume optimization.
- Tests on synthetic data do not prove the model generalizes or that an autonomous tuning loop improves performance.

Acceptance evidence requested from Windows

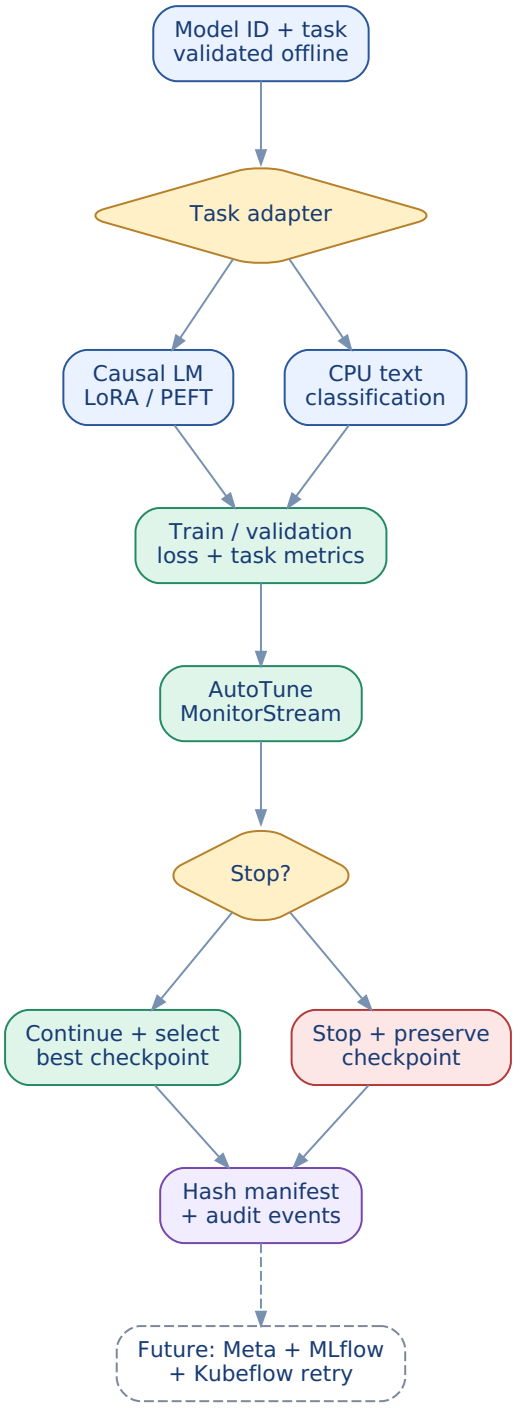
Send test count; preflight plan (`device: cpu`); actual train output; best eval loss; checkpoint manifest/adapter reload result; any traceback. Do not share access tokens.

9. Phase 5C: Universal Hugging Face open-source training (v0.8.0)

> **Latest verified implementation checkpoint:** 153/153 offline tests passed in the build environment, including 18 added classification task-validation, provenance and artifact-integrity tests. Two Hugging Face task adapters are implemented, but **live Transformer/PEFT training was not performed in this build** because the optional libraries and model downloads were not available. The earlier Phase 5A real GRU CPU run remains user-verified. Do not describe 153 offline tests as actual BERT or GPT-2 fine-tuning.

9.1 Why use a task router?

Hugging Face hosts many *different* architectures. GPT-style causal language models predict the next token, whereas BERT-style encoders can learn supervised class labels. Their data collators, heads, metrics and output artifacts are different. The correct universal interface is one **validated request contract** with separate task-specific training adapters, not a generic script that assumes any checkpoint can be trained as a causal LM.



Hugging Face CPU-first task adapters and autonomous monitoring boundary

Training mode	Source file	Example repository	Outputs	Current status
Causal language model / LoRA	`autotune/lora_gpu.py`	`sshleifer/tiny-gpt2`	PEFT adapters, Trainer checkpoints, JSONL metrics	Source + offline tests; live run pending
Text classification / full fine-tune	`autotune/hf_classification.py`	`google/bert_uncased_L-2_H-128_A-2`	Model weights, label mapping, Trainer checkpoints, JSONL metrics	Source + offline tests; live run pending

Training mode	Source file	Example repository	Outputs	Current status
Vision / audio / sequence-to-sequence	Future adapters	ViT / Whisper / T5	Task-specific	Not implemented

License distinction: publicly downloadable models are not automatically open source. Inspect the repository's license, usage restrictions, safety notices and gated-access requirements before training or distributing any derivatives.

9.2 Architecture and interfaces

The new `autotune/hf_models_cli.py` detects the task from YAML. With no flags, it **only performs offline preflight**, hashing the actual dataset files, checking dataset separation, model IDs, labels, parameter ranges and evaluation schedule. No model weights are downloaded at preflight.

For `causal-lm`, the CLI routes to the existing PEFT LoRA pipeline and verifies adapter manifests. The model's `hf.target_modules` are architecture-specific; blindly applying `c_attn` from GPT-2 to a BERT model will fail. The CLI limits the CPU LoRA path to configurations explicitly marked as CPU and rejects 4-bit quantization without an appropriate GPU.

For `text-classification`, the CLI loads `AutoTokenizer`, `AutoModelForSequenceClassification`, and `Trainer` only after `--execute`. Its JSONL rows must have a nonempty `text` and a string/integer `label`. The training split determines the label-to-index mapping. Validation labels must all appear in training, and duplicate text across splits is rejected. Labels and held-out records should represent a genuine task, not a tiny synthetic quality benchmark.

The classification callback logs training losses, validation loss and accuracy, then passes **paired training/validation observations** into the existing `MonitorStream`. Monitoring can request an early stop, but the agent does not yet approve a patch or schedule another real HF attempt. The output contains a saved best model, complete Trainer checkpoint metadata, run receipt and SHA-256 artifact manifest.

9.3 Windows PowerShell commands (no dedicated GPU)

From the extracted project root:

```
py -3.11 -m venv .venv-hf
.\.venv-hf\Scripts\python.exe -m pip install -r requirements-lora.txt
.\.venv-hf\Scripts\python.exe -m pytest -q

# Offline config + dataset preflight (no download)
.\.venv-hf\Scripts\python.exe -m autotune.hf_models_cli --config config/hf_bert_tiny_cpu.yaml

# Explicit CPU fine-tuning (downloads a small public model)
.\.venv-hf\Scripts\python.exe -m autotune.hf_models_cli --config config/hf_bert_tiny_cpu.yaml --execute
        --output outputs/bert-tiny-cpu-01

# Offline artifact-integrity verification
.\.venv-hf\Scripts\python.exe -m autotune.hf_models_cli --config config/hf_bert_tiny_cpu.yaml --verify
        outputs/bert-tiny-cpu-01/best_model
```

For causal language modeling, substitute `--config config/lora_public_cpu.yaml` in the first two run commands; it uses PEFT LoRA on `sshleifer/tiny-gpt2`. The existing `autotune.lora_cli` remains supported. An initial model download requires access to Hugging Face unless artifacts are already locally cached. Training on real user data needs consent, privacy checks, sensible split sizes, task-specific metrics, and independent test evaluation before promotion.

9.4 How an external Hugging Face repository is handled

Stage	Guardrail
Select	Verify repository ID, task/model compatibility, license and access
Validate	Enforce config limits, independent train/validation splits, stable label map
Acquire	Explicit `--execute`; `trust_remote_code=False`; prefer pinned model revision
Fine-tune	Per-task Trainer API, CPU budget, bounded steps and evaluation frequency
Monitor	Window-aligned training/validation metrics, conservative stopping
Recover	Integrity hash of checkpoints; explicit compatible resume; no automatic rewrite yet
Promote	Not implemented: future untouched test set and human/policy approval

9.5 Known limitations and implementation work ahead

This version does **not** implement vision, speech, token labeling, automatic dataset downloading, broad model-family compatibility detection, hosted Meta agent calls within HF training, MLflow tracking server integration, or Kubeflow-managed retries. It does not report a live BERT/LoRA training benchmark. The sample dataset has only 16 combined records and is a plumbing demonstration, not a

representative learning task. The standalone classification model stores full model weights, while the causal-LM adapter stores PEFT LoRA weights. Their checkpoints are not mutually interchangeable.

Next acceptance gate: run CPU BERT classification and tiny GPT-2 LoRA once on the user's Windows machine; check actual losses, reload/verify artifacts, and exercise an explicitly selected full Trainer checkpoint resume. Then connect both task adapters to the durable Phase 3 state machine, Meta recommendations, and Phase 6 MLflow tracking. Do not automatically requeue real training jobs until their termination and resume semantics are independently validated.

Further implementation instructions and JSONL schemas: `docs/HF_OPEN_SOURCE_MODELS.md`.

Appendix A - Unsloth integration

Version 0.8.3 | Optional GPU backend, CPU-first development preserved

Decision and current capability

Yes: Unsloth can join AutoTune AI as a selectable backend. It complements, rather than replaces, Hugging Face Transformers + PEFT. Unsloth is well suited to accelerated fine-tuning on supported NVIDIA or ROCm GPU hosts, subject to the model and driver compatibility matrix.

Current CPU-only host: the verified Hugging Face + PEFT LoRA path remains the default. Unsloth CPU inference and data tooling should not be conflated with Unsloth GPU-accelerated training. The new adapter refuses training when the GPU check fails.

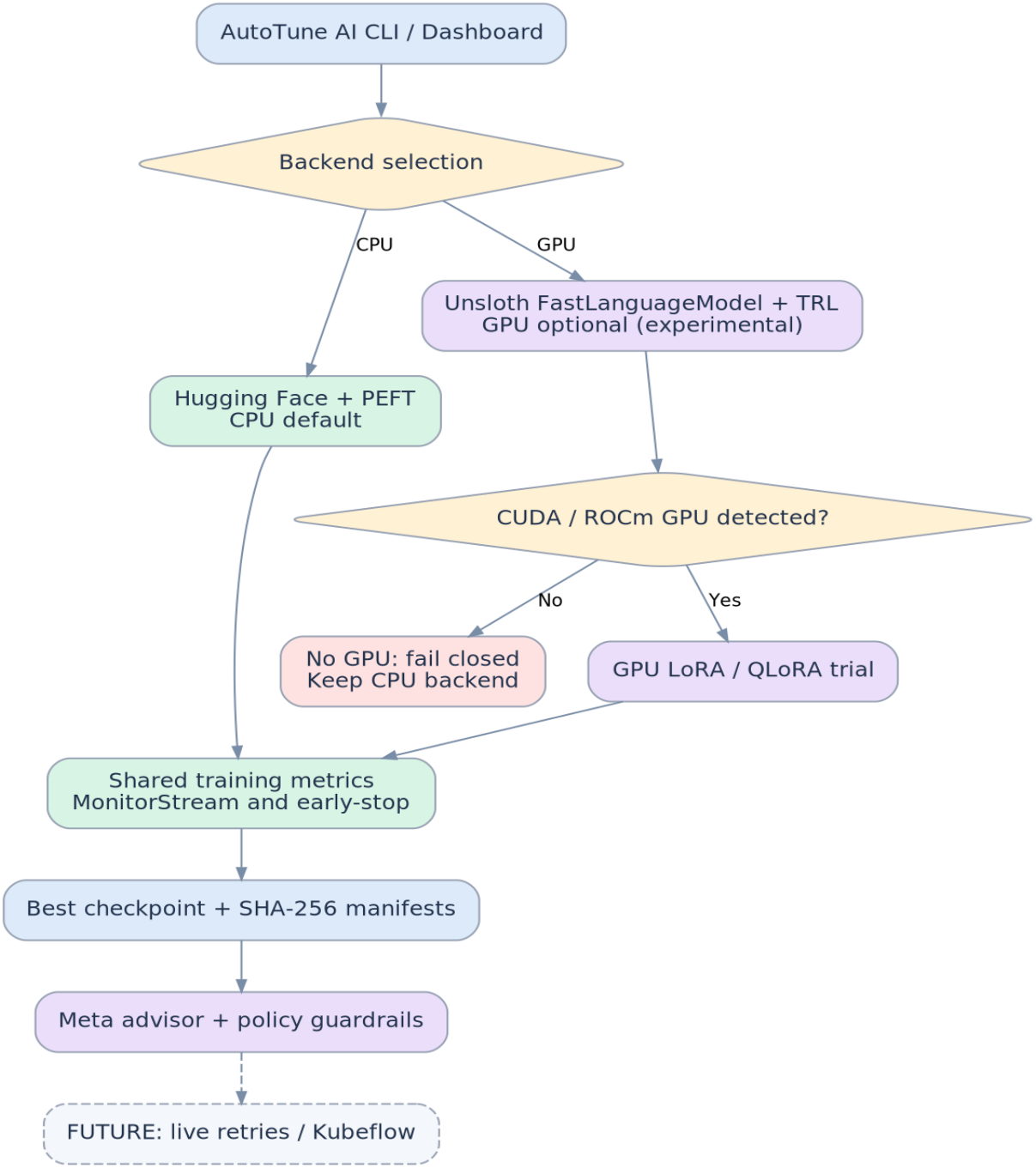
Integration matrix

Capability	Current status
HF + PEFT CPU LoRA	User-verified real Tiny GPT-2 fine-tune and fresh adapter reload
Unsloth offline preflight	Implemented and tested without importing Unsloth
Unsloth GPU SFT	Experimental source path; not live GPU-tested
Shared training monitoring	Callback implemented; actual Unsloth GPU runtime pending
Artifact integrity	Manifest hooks implemented; GPU checkpoint outputs pending
Agentic GPU retry / Kubeflow	Not connected; future gated milestones

Verified locally: 193/193 automated tests passed (185 inherited + 8 new). A CPU Unsloth training attempt was blocked before downloading weights or creating output files. This does NOT prove the GPU training path.

Appendix B - Backend architecture

The existing deterministic health monitor and checkpoint audit remain shared contracts. Unsloth is selected explicitly; no automatic fallback or hidden model download.



Control boundary: The Meta reasoning agent produces recommendations only. A deterministic policy engine must authorize changes before any future automatic restart. This experimental Unsloth path currently executes at most one trial and does not resume optimizer state.

Appendix C - Running and validating

Windows PowerShell | Safe CPU preflight and optional GPU deployment checklist

Safe commands on your current CPU Windows setup

From the existing project root, after applying the v0.8.3 update patch:

```
python -m autotune.unsloth_cli
--config config/lora_unsloth_gpu.example.yaml
python -m autotune.unsloth_cli
--config config/lora_public_cpu.yaml
python -m pytest -q
```

These commands check configuration only; no model is downloaded or trained. **Do not use the Unsloth `--execute` flag on this CPU machine.** Continue the verified HF PEFT workflow.

GPU execution acceptance gates

• Provision a compatible GPU and separate environment; follow current Unsloth NVIDIA/AMD installation requirements rather than modifying the working CPU .venv.

• Pin model revision and dependency versions, validate target modules, model license/access, and actual PyTorch CUDA/ROCm capability.

• Execute one short real GPU SFT with measured train/validation metrics, checked checkpoint artifacts and a fresh adapter reload.

• Prove full Trainer optimizer/scheduler resume and early-stop semantics, then connect bounded agent recommendations and orchestration after an independent end-to-end test.

Next priority in the CPU-first plan

Complete the existing v0.8.2 isolated Hugging Face paused-run resume exercise. Then connect the actual HF training loop to the durable policy engine. Unsloth GPU execution can be tested later on compatible remote compute; do not gate CPU development on it.

References

Unsloth project: github.com/unslothai/unsloth | GPU requirements: unsloth.ai/docs/get-started/fine-tuning-for-beginners/unsloth-requirements | AMD install: unsloth.ai/docs/get-started/install/amd | Windows install: unsloth.ai/docs/get-started/install/windows-installation

Real Hugging Face Self-Healing

AutoTune AI v0.8.6 | 2 November 2026 | CPU-first Windows implementation

Executive checkpoint

This release connects the existing real Hugging Face LoRA trainer to a bounded agentic recovery controller. It uses genuine training and validation measurements and verified Hugging Face checkpoint artifacts. It can propose and apply restricted numeric hyperparameter patches as isolated **new training trials**.

Evidence	Result
Current executable source	v0.8.6
Local automated tests	208 passed (193 inherited, 15 new)
Controller test technique	Fake Trainer boundary + real SHA-256 artifact manifests
Previous Windows confirmation	192 passed, 1 skipped on v0.8.3
Live Phase 5E real two-attempt CPU run	Not yet executed/verified on Windows
Meta hosted API / MLflow / Kubeflow	Not live verified

What changed since v0.8.5

• New `autotune/hf_selfheal.py`: offline preflight and explicit real Hugging Face training mode.

• Real Trainer callback now exposes a test-only, explicitly labeled forced-stop signal.

• The controller versions configuration YAML, checks provenance and checkpoint hashes, and writes an audit journal.

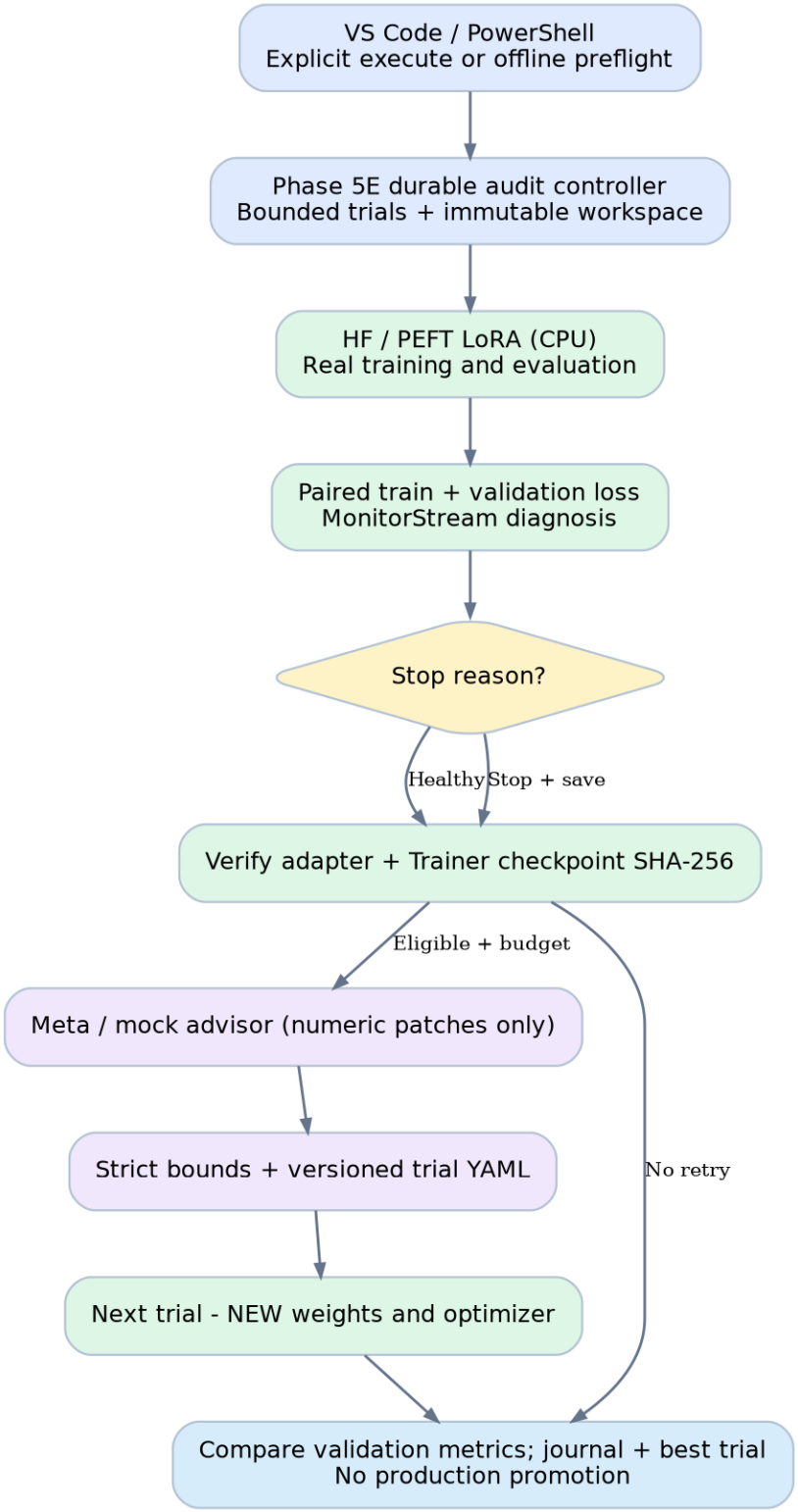
• New sample CPU configuration and 15 controller safety/integration tests.

Critical distinction: A fault-injected stop is not proof that overfitting naturally occurred. The integrated process still requires a live Windows acceptance run.

SYSTEM ARCHITECTURE

Phase 5E integration boundary

All trials are executed one at a time. A new trial starts with fresh model/optimizer state when hyperparameters change. Advisor suggestions are validated independently and never receive shell or arbitrary code-edit authority.



Source: `docs/diagrams/hf_selfheal_v086.mmd`. The diagram is a proposed operational flow; live end-to-end behavior still awaits the Windows run.

TECHNICAL DESIGN

Controller lifecycle and safeguards

Stage	Implementation and intended guarantee
Preflight	Validates CPU model config, dataset split checksums and retry budget without downloads.
Start trial	Writes unique `trial-0N.yaml` and a `controller_state.json` receipt.
Monitor	Trainer callback reports real train/validation metrics to `MonitorStream`; only eligible stop states can trigger a retrial.
Verify	Hash-checks full Trainer checkpoint (including optimizer/scheduler/RNG) and final adapter.
Advisory	Meta or mock advisor returns a patch; policy restricts keys, ranges and movement direction.
Retry	Creates a fresh training trial with changed YAML; does not misuse optimizer-state resume.
Finalize	Compares recorded validation losses; retains both attempts and `controller_result.json`.

Safety boundaries

• A maximum of three total attempts; example configuration limits to two.

• Nonempty experiment workspaces are rejected; finished historical outputs are never overwritten.

• Only stable `overfit` and `underfit` diagnoses permit automatic hyperparameter edits. Divergence does not.

• Model, revision, data and device remain fixed; each trial has its own configuration fingerprint.

• Unexpected failures mark a run **blocked**. There is no automatic crash recovery or remote-job reconciliation yet.

CONTROLLER ACCEPTANCE TEST

Controlled early-stop exercise

A tiny 4-train/2-validation sample cannot reliably produce meaningful generalization behavior. To test the full handoff, `--fault-inject-overfit` requests a **clearly labeled synthetic stop** at the first real evaluation. The measured losses remain genuine, and the synthetic trigger is recorded separately.

Action	Expected evidence
Trial 1 trains on CPU	Actual HF Trainer updates and evaluation metrics.
Injected stop at eval	`fault_injection_stop`, `synthetic_trigger: true`, preserved checkpoint.
Controller requests policy-safe patch	`trial-01.advisor_decision.json`, patch keys restricted.
Trial 2 starts fresh	`trial-02.yaml` and separate Trainer output.
Both trials checked	Manifests for full checkpoints and adapters verify.
Final selection	`controller_result.json` records attempts and best validation loss.

This does not prove

- That the first model naturally overfit, or that the patched model generalizes better.
- Production self-healing under crashes or distributed runtime failures.
- Optimizer-equivalent resumption between patched trials: these deliberately start fresh.

Run in VS Code PowerShell

Apply the v0.8.6 update patch to your source folder; preserve `.venv` and all prior `outputs`. Work from the repository root.

1. Verify the environment and preflight

```
.\.venv\Scripts\python.exe -m pytest -q

.\.venv\Scripts\python.exe -m autotune.hf_selfheal --config config/lora_selfheal_cpu.yaml
```

2. Execute the real, controlled two-trial CPU test

```
.\.venv\Scripts\python.exe -m autotune.hf_selfheal --config config/lora_selfheal_cpu.yaml --execute
--fault-inject-overfit --workspace outputs/hf-selfheal-injection-01
```

3. Inspect receipts, metrics and recommendation

```
Get-Content outputs\hf-selfheal-injection-01\controller_result.json -Raw

Get-Content outputs\hf-selfheal-injection-01\controller_events.jsonl

Get-Content outputs\hf-selfheal-injection-01\trial-01.advisor_decision.json -Raw
```

4. Earlier Phase 5D checkpoint evidence

```
Get-Content outputs\gpt2-resume-probe-01\resume_proof.json -Raw
```

Expected result: an explicit fault-injection receipt, two isolated real Hugging Face attempts, verified manifests, and an auditable best-trial comparison. Use a new `--workspace` for every repeat.

VALIDATION AND NEXT PHASE

Acceptance gates and project roadmap

Gate	Status
208 local Python tests on source v0.8.6	PASS
Real two-attempt HF CPU execution on Windows	PENDING
Live, naturally diagnosed early-stop condition	PENDING
Optimizer/scheduler fields in older resume_proof.json	USER EVIDENCE PENDING
Meta hosted model API diagnosis	NOT VERIFIED
MLflow tracking server / Kubeflow / Katib	NOT VERIFIED
Independent held-out evaluation and promotion	NOT IMPLEMENTED

Recommended continuation

• **First:** run the controlled two-attempt CPU experiment; compare the two versioned training configurations and verified manifests.

• **Then:** obtain naturally diagnosed overfit/underfit evidence on realistic datasets with holdout evaluation, without forcing a stop.

• **Next:** integrate durable MLflow run lineage and persistent artifact storage; add CPU-native Kubeflow/Katib orchestration afterwards.

• **Optional later:** benchmark Meta Llama or Unsloth on remote, supported GPU hardware.

Project continuity: AGENTS.md is the authoritative progress record; the master execution plan contains future dependencies and acceptance criteria. Every test claim must retain its verification boundary.

Verified real recovery; optional MLflow lineage

November 2, 2026. Phase 5E was independently verified on Windows with **207 passing tests, 1 skipped**, followed by two controller-managed **real Tiny GPT-2 LoRA CPU attempts**. Trial 1 stopped under an **explicitly injected** overfitting event; a validated configuration change produced trial 2, which completed. This is a functional acceptance test, not evidence of naturally detected overfitting.

Verification item	Observed evidence
Trial 1 validation loss	10.365184783935547
Trial 2 validation loss	10.365181922912598
Selection	Trial 2; loss difference approximately 0.00000286
Deployment	promotion_approved = false
Meta advisor	Mock provider; hosted Meta API not live-verified
Phase 6A offline tests	216 passing (208 previous + 8 additional); no live MLflow server test

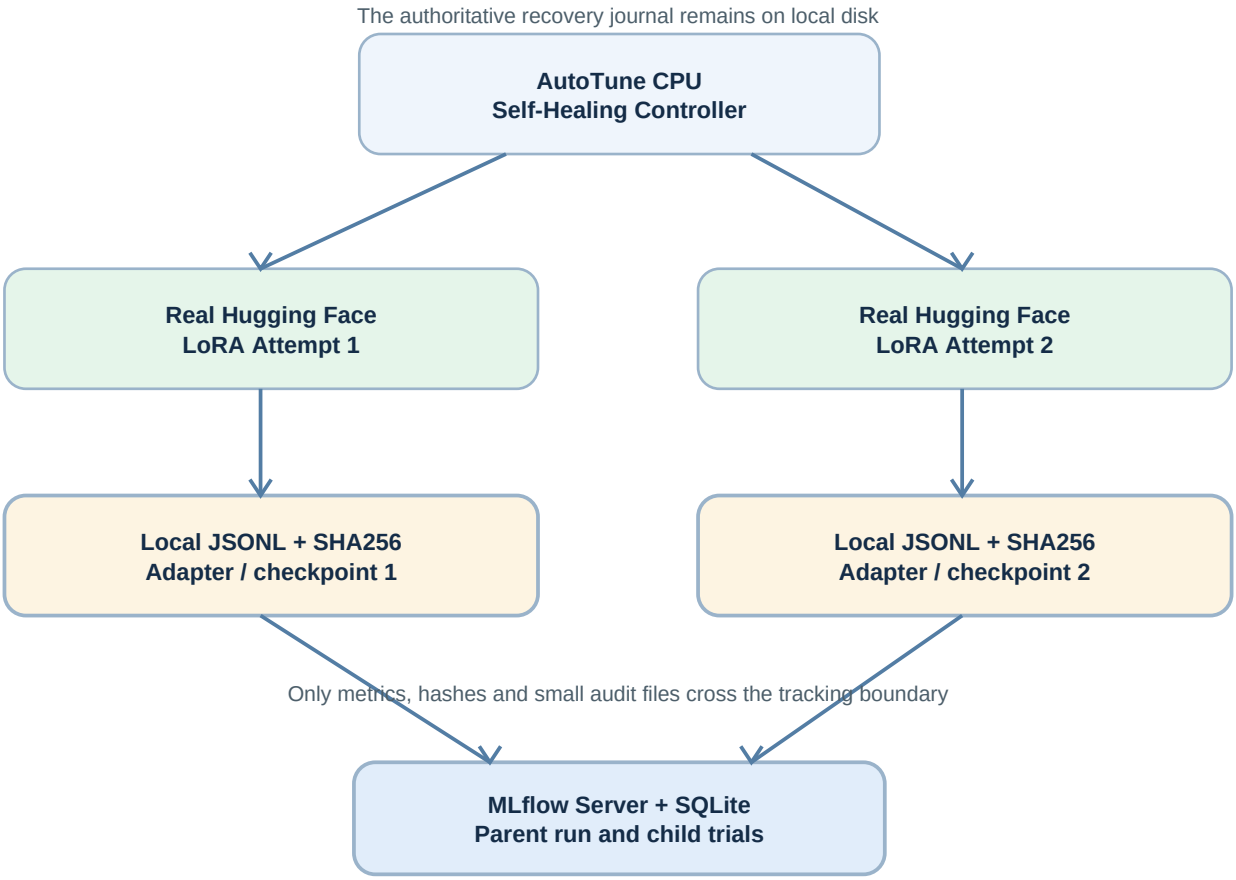
Implementation summary

Phase 6A introduces **autotune/hf_mlflow.py** and opt-in flags on the established **autotune.hf_selfheal** entrypoint. It records experiment parent-child lineage, per-step metric records after each attempt, config hashes, diagnosis and fault-injection provenance, best-trial metadata, and small integrity manifests.

Boundaries: No MLflow import or network activity unless the user passes --mlflow-uri. No .pt or .safetensors model weight upload. A failed explicitly enabled tracking operation blocks the controller with a local audit record rather than silently claiming success. Live MLflow server and UI verification remains pending.

Architecture and traceability

One controller, multiple real training attempts, and a separate optional tracking plane.



Controller run and child trial runs

The parent run stores dataset/config hashes, provenance, best-trial selection and non-promotion status. Each child run stores training hyperparameters, scalar train/validation metrics, the observed diagnosis, whether fault injection occurred, and the checkpoint integrity manifests.

What is intentionally not part of this release

MLflow is not yet used as an authoritative checkpoint store or model registry. No artifact-weight transfers or deployment approvals occur. The reporter currently uploads validated metrics after each attempt; continuous live metric streaming is reserved for Phase 6B.

Windows PowerShell verification runbook

Apply the update patch to your existing VS Code project. Keep all previous outputs, especially the successfully verified two-attempt run. Use a new, empty workspace for every MLflow acceptance test.

1. Install the optional dependency and verify the tests

```
.\.venv\Scripts\python.exe -m pip install -r requirements-mlflow.txt

.\.venv\Scripts\python.exe -m pytest -q
```

2. Start a local tracking server in Terminal A

```
.\.venv\Scripts\mlflow.exe server --backend-store-uri sqlite:///mlflow.db --host 127.0.0.1 --port 5000
```

Open **http://127.0.0.1:5000** locally. The server uses SQLite and does not require Docker or a GPU. Keep this terminal running. Official reference: https://mlflow.org/docs/latest/tracking/server/.

3. Start a new CPU acceptance run in Terminal B

```
.\.venv\Scripts\python.exe -m autotune.hf_selfheal --config config/lora_selfheal_cpu.yaml --execute
--fault-inject-overfit --workspace outputs/hf-selfheal-mlflow-01 --mlflow-uri http://127.0.0.1:5000
--mlflow-experiment AutoTune-AI-CPU
```

4. Acceptance checklist

Verify **ok: true**, two attempts, no promotion, one controller parent run and two linked child runs in MLflow, scalar metrics in the trials, and controller outcome in the parent. Confirm the MLflow artifact view contains only small YAML/JSON/JSONL metadata, not model weights. Cross-check local controller_events.jsonl for the MLflow run IDs.

Roadmap following the acceptance test

Phase 6B: live metric streaming, read-only comparisons, failure reconciliation and exports. **Phase 7:** local CPU Kubernetes/Kubeflow orchestration. **Phase 8:** Katib trial optimization. The hosted Meta advisor and GPU-specific Unsloth path are still separate unverified integrations.

Frontend and GPU strategy

Architecture decision: TypeScript frontend; GPU-neutral Python training backends

Decision 1 | Next.js + TypeScript is the primary platform UI

Streamlit is optional for developer-only experiments. Next.js provides typed API contracts, routing, reusable controls, authentication integration and a production-friendly multi-user interface. Keep training, agents, and orchestration in Python/FastAPI.

Decision 2 | GPU profiles are selectable, not hardcoded

Profile	Example hardware	Positioning
CPU only	**No GPU**	Current Windows development, Tiny GPT-2 / tiny BERT, monitoring and MLflow
Entry CUDA	**RTX 5060 Ti 16 GB**	Small/medium LoRA; QLoRA subject to actual sequence and memory profile
Practical CUDA	**RTX 3090 / 4090 24 GB**	More headroom for LoRA experiments; check used-hardware condition
Premium CUDA	**RTX 5090 32 GB**	Higher-capacity local experiments; verify driver and Unsloth compatibility
AMD cloud	**MI300X 192 GB**	ROCm on Linux; large-model training and multiple experiments
AMD scale	**MI325X 256 / MI350X 288 GB**	High-memory data-center workloads; enterprise infrastructure

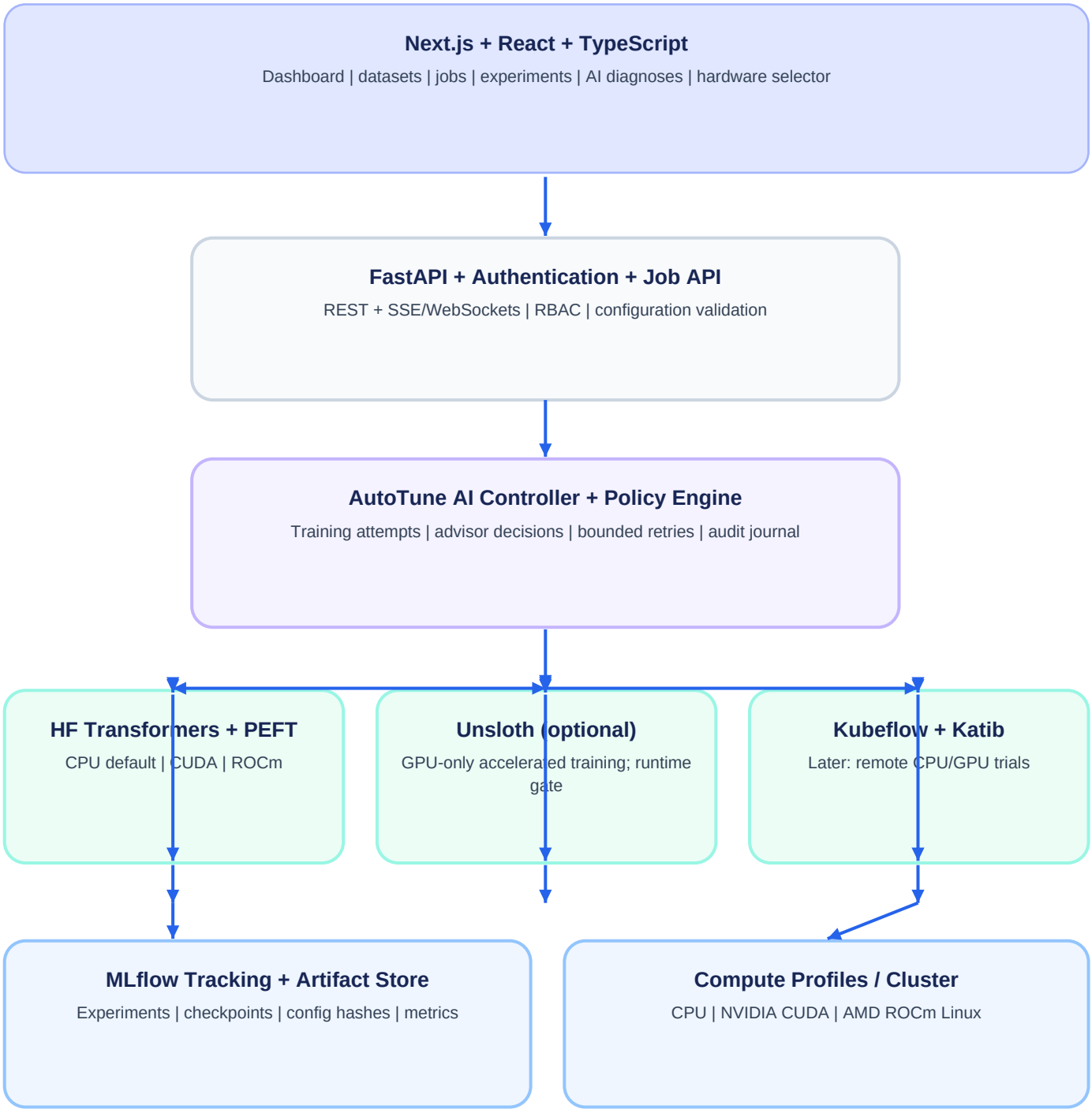
Recommended selection for the next sprint

Do not buy a GPU just to build the UI. Finish the CPU-based MLflow tracking acceptance test and TypeScript dashboard first. When hardware is available, start with a 16-24 GB NVIDIA CUDA device for straightforward Unsloth smoke tests, or use a remote AMD MI300X running a supported Linux/ROCm stack.

GPU memory is not a guarantee of model fit: optimizer, activations, context length, batch size, quantization, attention backend and concurrent jobs all affect requirements.

Proposed integrated architecture

TypeScript controls jobs; Python owns training; backends select CPU / CUDA / ROCm



Training jobs never execute in the browser. The UI submits a validated training specification and receives an immutable job ID; Python workers own checkpoints, agent decisions and GPU selection. Client code must not contain Hub tokens, Meta API keys or storage credentials.

Implementation and acceptance gates

No source-code changes in this documentation checkpoint

TypeScript dashboard MVP

- Create /frontend using Next.js App Router, TypeScript, Tailwind and shadcn/ui.
- Expose FastAPI routes: GET /health, GET /experiments, POST /training/jobs, GET /training/jobs/{id}, GET /training/jobs/{id}/events.
- Model the job spec with typed fields: task, model_id, dataset hashes, backend, device profile, retries and HPO budget.
- Display experiment list, training curves, best validation metrics, stop cause, approved parameter changes and artifact checks.
- Support explicit fault-injection badges and never label it natural overfitting. Include cancellation and audit-log views.

GPU capability-detection gates

- CPU: default to existing Hugging Face PEFT runner; no accelerator required.
- NVIDIA: require available CUDA device, package/driver compatibility, available VRAM, and trial training smoke test.
- AMD: determine gfx target, ROCm/PyTorch compatibility, available VRAM and backend support; use Linux for MI300/MI350.
- Unsloth: executable only after model/license, device, backend and version checks pass; GPU execution remains unverified.
- Resource quota: enforce max attempts, GPU-minutes, memory and per-user concurrency before starting a trial.

Source and acceptance evidence

Next.js TypeScript docs: https://nextjs.org/docs/app/api-reference/config/typescript
Unsloth requirements: https://unsloth.ai/docs/get-started/fine-tuning-for-beginners/unsloth-requirements
AMD + Unsloth support: https://www.amd.com/en/developer/resources/technical-articles/2026/train-and-run-models-on-amd-gpus-with-unsloth.html
AMD GPU family capacities: https://rocm.docs.amd.com/projects/ai-ecosystem/en/latest/optimization/workload-optimization.html
NVIDIA RTX 50 specs: https://www.nvidia.com/en-in/geforce/10-series/10-series-specs/

Status: Phase 5E real CPU two-attempt fault-injection loop verified by user; Phase 6A MLflow adapter passed offline tests, live server acceptance pending. TypeScript UI and GPU runs are planned, not implemented in this update.

PHASE 6A / WINDOWS LIVE ACCEPTANCE

MLflow tracking is now verified

Real Hugging Face PEFT LoRA training completed twice under the CPU self-healing controller, with three MLflow tracking run links issued by the localhost server.

Acceptance check	Observed evidence	Interpretation
Tracking connection	3 MLflow run links (controller + trial-01 + trial-02)	Server connection and run creation verified
Attempt count	2 completed real CPU attempts	Bounded real training and retry verified
Run workspace	outputs/hf-selfheal-mlflow-02	Fresh path; previous results preserved
Final controller status	ok=true; status=completed; best_trial=2	Control loop completed
Release gate	promotion_approved=false	No unsupported model promotion

Observed loss and fault semantics

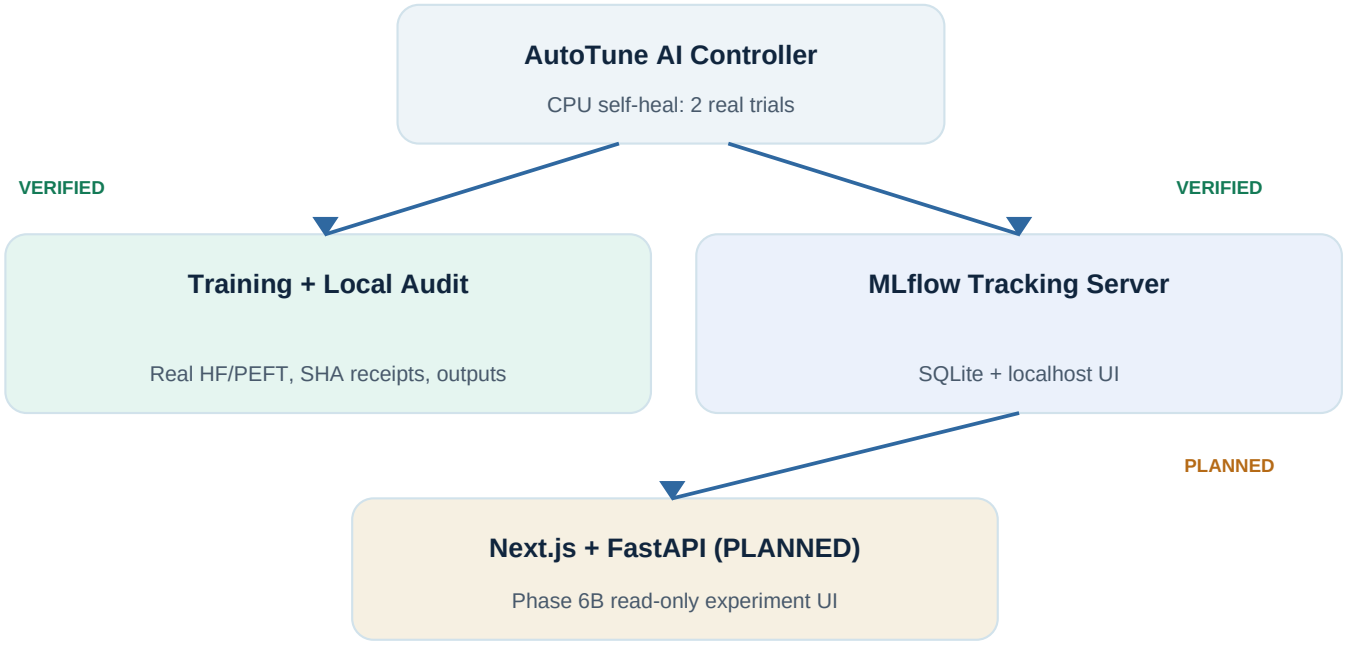
Attempt	Best validation loss	Trigger
Trial 1	10.365182876586914	Explicitly injected overfit
Trial 2	10.365181922912598	No stop diagnosis
Difference	0.000000953674316	Not a meaningful improvement

Interpretation: This is evidence that recovery orchestration and MLflow tracking can work together on a real CPU model. It is not proof of naturally detected overfitting, improved model quality, or production readiness. The demo uses only four training and two validation records.

Pending MLflow UI proof: Inspect recorded parent-run tags, child links, metric histories and audit artifacts in the MLflow UI / API. Console run links alone do not establish the correctness of all visual relationships.

Experiment tracking architecture

Durable local checkpoint records remain authoritative. MLflow is an optional tracking and visualization layer, not the controller's sole state store.



Evidence per run

Controller run ID: `f6c7c00ccfed4a5b80449f22cff972df`

Trial 1 run ID: `dca1c1ac15d8498991fc40b2ddc98d6f`

Trial 2 run ID: `7f914d30a4214e4f9b15841453e341f7`

The MLflow host is **127.0.0.1:5000** on the user's own Windows machine, not a public server. The assistant has not opened that private UI.

Source baseline: executable v0.8.7; documentation v0.8.8 defined the TypeScript dashboard architecture; v0.8.9 adds the live MLflow acceptance record only. No new executable source or tests were produced in this addendum.

EXECUTION PLAN / NEXT MILESTONE

Phase 6B: read-only dashboard

Build the monitoring experience on top of proven local training and tracking before introducing new job-execution surfaces. The frontend remains CPU-only and does not need Docker.

Step	Build task	Done when
1. API	FastAPI GET /health, /experiments, /runs/{id}, /runs/{id}/metrics	Unit tests plus localhost MLflow integration
2. Frontend	Next.js App Router + TypeScript: experiment list, run detail, charts	TypeScript build passes; no GPU needed
3. Audit labels	Show injected vs genuine stops, config diff, best trial and promotion status	No mislabeling of synthetic events
4. Tracking quality	Resolve run hierarchy from MLflow metadata, not naming conventions	Parent-child links and metrics confirmed
5. Safety	Read-only endpoints, bounded responses, secrets never returned, path traversal rejection	Negative tests pass

Acceptance questions to close

- Does the MLflow UI show one controller run and two linked attempt runs?
- Can the dashboard show metric sequences and correctly label the injected overfit?
- Can the read-only API operate with MLflow temporarily unavailable and display an explicit error?
- Are all past output directories untouched and does the promotion gate remain false?

Outstanding separate proof: inspect the existing resume_proof.json content for optimizer/scheduler restoration. Hosted Meta API, production Kubeflow/Katib, and Unsloth GPU training remain future integrations.

Security Across the Training Lifecycle

Documentation checkpoint v0.8.10 | Source: v0.8.7 | Status: design and source review, not a deployed vulnerability scanner

Purpose. Show precisely how AutoTune AI currently detects training faults and enforces integrity/policy controls, and which security-specific vulnerability detectors must be added. A training anomaly is not automatically proof of an attack.

Threat and evidence model

Issue	Representative signal	Security conclusion
Overfit, divergence, gradient spike	Train/validation loss trends and nonfinite metrics	Training fault or anomaly; not proof of poisoning or compromise
Poisoned dataset or model	Anomalous samples, suspicious labels, targeted behavior	Needs separate provenance + poisoning + robustness tests
Supply-chain compromise	Vulnerable dependency, unsafe pickle, mutable model revision	Requires advisory scan, pinned immutable revision, safe-load policy
Privacy exposure	PII in training data/logs, memorization in model output	Requires PII scanning and privacy/red-team evaluation
Artifact tampering	Hash/provenance mismatch vs trusted manifest	Detects modifications; cannot detect a malicious original artifact

Current evidence boundary

Verified on user Windows CPU: real Hugging Face LoRA training, adapter audit and reload, controlled pause/resume, two real controller-managed attempts using fault-injected overfit, and local MLflow run creation. The project does not yet demonstrate authentic poisoning detection, automated CVE/PII scanning, adversarial test suites, authenticated deployment, or a hosted Meta advisor call.

A fault-injected diagnosis tests orchestration behavior only; it should never be reported as discovery of a real security vulnerability.

Lifecycle security gates

Stage	Current AutoTune AI behavior	Next required security detector
1. Ingest	JSONL format, nonempty text, same-split duplicate and cross-split overlap checks; dataset SHA-256.	PII/secret inspection, label quality, anomaly triage, provenance and license intake.
2. Model source	Model name/revision recorded; adapter/checkpoint manifest verification.	Pin immutable commit SHA; trusted allowlist, signed manifest review, safetensors-only base model policy where supported.
3. Dependencies	Separate requirements; Python tests and CPU preflight.	pip-audit + SBOM, fixed versions, secret scanning, SAST/CI policy.
4. Live train	Monitor train/validation losses, grad norms and invalid events; stop/continue decisions.	Security event stream, resource abuse indicators, anomalous batch attribution and poisoning investigations.
5. Agent recovery	Numeric-only advice, strict JSON, policy bounds, retry budget, audit records.	Prompt-injection red teaming, denial escalation, path/network restrictions for future APIs.
6. Artifacts	SHA-256 manifests, separate attempt folders, safe-write/locking checks.	Trusted provenance roots, signature validation, quarantine of hostile deserialization files.
7. Evaluation	Best validation loss selected, deployment promotion false.	Robustness tests, backdoor/trigger checks, output memorization/PII leakage tests, held-out security suite.
8. Tracking and deployment	Local MLflow accepted parent/child runs; small artifact logging; no default model-weight upload.	Authenticate shared MLflow, secrets management, role permissions, audit retention, deployment approval/rollback.

Policy: alert, quarantine, stop or retry?

Training instability: gracefully stop, preserve the best trusted checkpoint, and consider a bounded retry only after configuration validation. Suspected security compromise: stop and quarantine the run and its inputs; investigate or require human approval. Never let a generative model silently classify a suspected compromise as a routine hyperparameter issue.

Detection and response workflow

Step	Decision	Action and evidence
01	Preflight gate	Validate JSONL and split fingerprints. Planned: scan PII, malicious content, dataset provenance, model source and dependency advisories.
02	Training observer	Evaluate aligned training/validation metrics, nonfinite events, gradient norms, and resource usage. Create a distinct security-event channel.
03	Classify	Training fault vs suspicious security behavior. Do not treat a single noisy loss spike as attack evidence.
04	Remediate	Training fault -> bound hyperparameter changes and retry. Security suspicion -> block/quarantine and notify reviewer.
05	Validate artifacts	Hash checkpoint and adapter manifest; confirm trusted origin and absence of unsafe executable serialization.
06	Release gate	Run held-out accuracy/robustness/privacy tests; require promotion policy and explicit approval before deployment.

Example: how loss monitoring differs from poisoning detection

Suppose training loss decreases while validation loss rises for multiple evaluations. The existing monitor may flag overfitting and recommend early stopping. But it cannot say whether the cause is (a) too many epochs, (b) distribution shift, (c) label corruption, or (d) an intentional data-poisoning attempt. To assess poisoning, the next security layer must analyze sample provenance, label anomalies, suspicious clusters, and targeted test behavior before deciding to quarantine data.

Example: hash verification is not malware scanning

AutoTune AI compares stored checkpoint/adapter file hashes to a manifest. That reveals changes after the trusted manifest was created. A malicious model downloaded and hashed at intake could still pass. Mitigation requires trusted origin, immutable revisions, safe weight formats, review, and isolation before loading.

Security logging schema (proposed)

```
event_time_utc | run_id | attempt_id | stage | detector | threat_category | severity | evidence_sha256 | decision | reviewer | status
```

Do not write raw secrets, sensitive dataset excerpts, or unrestricted user prompts into MLflow artifacts.

Priority implementation plan

Priority	Next action	Acceptance criteria
P0	Dependency audit and version pinning	Record pip-audit/OSV findings; fail on unreviewed critical items; publish a reproducible pinned dependency inventory.
P0	Trusted model + checkpoint import	Pin commit SHA; reject unsafe formats or remote code by default; never load untrusted pickle as part of ordinary inspection.
P0	Data and log privacy guard	PII/secret scan before train and before publishing MLflow artifacts; blocked disclosures generate redacted audit events.
P1	Poisoning and anomalous-label triage	Dataset provenance manifest, suspicion score, human review queue, labeled test fixtures.
P1	Agent safety adversarial testing	Test prompt injection, malformed JSON, oversized requests, parameter escalation, network/path abuse; all fail closed.
P1	Independent model-security evaluation	Backdoor probes, privacy memorization tests, robustness checks, test-set isolation and promotion block.
P2	Shared infrastructure hardening	MLflow auth/TLS/access policy; FastAPI input controls; Kubernetes RBAC/secrets/network policy before remote service.

Immediate read-only checks on Windows

From the project root using the existing Python environment:

```
.\.venv\Scripts\python.exe -m pip freeze > installed-packages.txt

.\.venv\Scripts\python.exe -m pip install pip-audit

.\.venv\Scripts\python.exe -m pip_audit
```

The first command records installed versions. pip-audit checks known dependency vulnerabilities; installing it changes the development environment, so do it in a dedicated security environment if reproducibility is paramount. Scans cannot prove the code or weights are safe.

Security architecture decisions and references

Decisions for AutoTune AI v0.8.10

1. Keep training anomalies and actual cybersecurity findings as distinct event classes. Never infer a security breach from loss curves alone.

2. Keep the Meta advisor limited to diagnostics and numeric recommendations; a deterministic policy controls hyperparameters, paths, credentials, retries, and deployment rights.

3. Preserve MLflow as observability and lineage, not the security enforcement authority. Do not upload raw training data or model weights by default.

4. Treat dataset/model/package intake and model promotion as security gates with fail-closed behavior; use quarantine and operator review for suspicious inputs.

5. Continue CPU-first. Do not require a GPU, Unsloth, Kubeflow or Katib to implement P0 scanners.

References used for this checkpoint

NIST AI 100-2 e2025 — Adversarial Machine Learning Taxonomy
https://csrc.nist.gov/pubs/ai/100/2/e2025/final

OWASP Secure AI Model Ops Cheat Sheet
https://cheatsheetseries.owasp.org/cheatsheets/Secure_AI_Model_Ops_Cheat_Sheet.html

OWASP ML Security Top Ten (draft)
https://owasp.github.io/www-project-machine-learning-security-top-10/

OWASP LLM04:2025 — Data and Model Poisoning
https://genai.owasp.org/llmrisk/llm042025-data-and-model-poisoning/

Hugging Face Hub — Pickle Scanning
https://huggingface.co/docs/hub/security-pickle

MLflow — Protect Your Tracking Server from Network Exposure
https://mlflow.org/docs/latest/self-hosting/security/network/

pip-audit — PyPI package and vulnerability scanner
https://pypi.org/project/pip-audit/

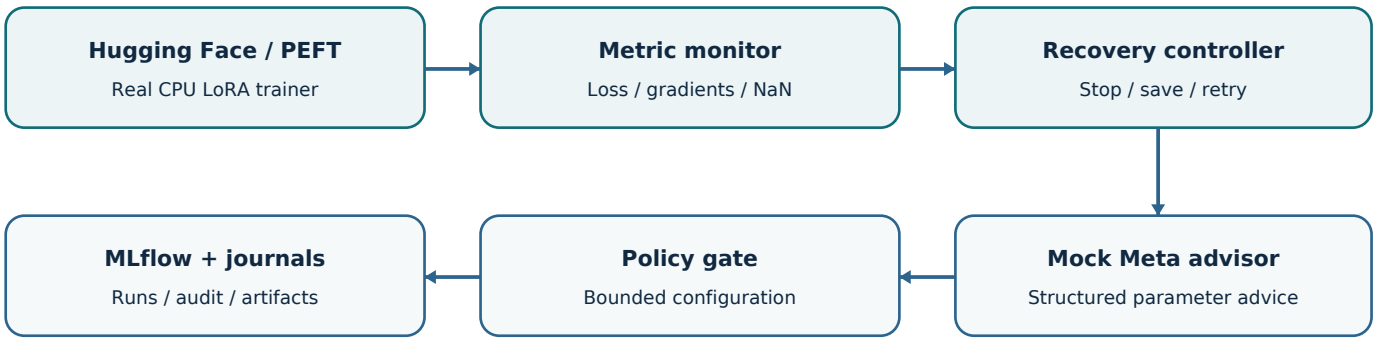
Evidence status: read-only review of the supplied v0.8.7 source ZIP plus the user-confirmed Windows logs. No security scanner, hosted service, penetration test, real poisoning benchmark, or new test suite was executed for this documentation update.

How I Implemented AutoTune AI

Student interview / project-viva narrative, matched to verified implementation evidence. No new source code or security scans were run for this documentation update.

1. Implementation overview

I built a CPU-first, self-healing training platform using Python, PyTorch, Hugging Face Transformers, PEFT LoRA, a rule-based training monitor, a bounded recovery controller, and MLflow. The AI-advisor interface is integrated and mock-verified; a live Meta inference call has not yet been verified.



Decision loop is real for a fault-injected training stop; cybersecurity scanners are a separate planned gate.

2. What actually detects a problem?

Signal	Detector / evidence	Action
Rising validation loss; falling train loss	Custom monitor: sustained overfit pattern	Stop if thresholds met
NaN, Inf, or repeated loss spikes	Training callbacks and health rules	Stop or flag instability
Unsafe parameter suggestion	Numeric-only advice + deterministic policy	Reject patch
Changed saved artifact	SHA-256 manifest vs expected fingerprint	Reject checkpoint
Attempt history	Local recovery journal and MLflow runs	Audit, compare, select

Viva answer: vulnerability tracking and correction

3. A concise answer to say aloud

"I distinguish training failures from security vulnerabilities. During training, a custom monitoring engine tracks training and validation loss, gradient norms, and non-finite metrics. When it identifies a sustained abnormal pattern, the controller stops the attempt safely, preserves its checkpoint, and requests numeric hyperparameter advice. A policy engine enforces limits before the next attempt starts. MLflow shows the training runs, parameters, metrics, and recovery outcome. To detect unintended artifact modification, we use SHA-256 validation and dataset/configuration fingerprints."

"For cybersecurity threats, the current system has defensive integrity and authorization boundaries, but it does not yet automatically scan for dependency CVEs, poisoned data, PII, or model backdoors. We plan to integrate pip-audit, trusted model revision policies, and dataset-security checks. For suspected security compromise, the correct response is quarantine and review, not blind retraining."

4. How we tested it

A real Windows CPU experiment fine-tuned **sshleifer/tiny-gpt2** using PEFT LoRA. A two-attempt controller workflow used an **explicitly injected** overfit stop for attempt one, then applied a bounded configuration update and completed attempt two. The local MLflow server accepted three run records (controller and two attempts). The result demonstrates working orchestration, not a real overfitting attack or meaningful model-quality improvement.

The user reported **207 tests passed, one skipped** in the Phase 5E source and verified real Hugging Face adapter reload; separate pause/resume testing progressed from training step 2 to step 4. The specific optimizer/scheduler resume proof JSON had not been independently inspected in conversation.

5. What is already present vs. what is still needed?

Already implemented / verified	Still planned or unverified
Loss and gradient monitoring; bounded retry policy	Automated CVE/SBOM scanning inside controller
Dataset schema, split overlap, fingerprints	PII, poisoned-label and backdoor detection
Hash-verified adapters and checkpoints	Trusted-signature / safe model-import gate
MLflow three-run tracking on localhost	Multi-user MLflow authentication and RBAC
Mock AI advice plus strict guardrails	Live Meta API diagnosis and adversarial audit
Real HF CPU training and fault-injected retry	Naturally detected end-to-end overfitting recovery

Questions an evaluator may ask

Q1. Does MLflow detect cybersecurity vulnerabilities?

No. MLflow is our experiment tracking and observability tool. It records metrics, trial parameters, and audit metadata. A dedicated scanner such as pip-audit is needed for known Python dependency vulnerabilities, while dataset poisoning and privacy need separate specialized tests.

Q2. What happens if your model is overfitting?

The monitor looks for a sustained pattern of falling training loss and increasing validation loss across evaluations. If a configured stop threshold is met, we preserve a checkpoint, record a diagnosis, accept only validated hyperparameter changes, and launch another versioned attempt. In the verified real HF self-healing demo, the trigger was injected on purpose.

Q3. Why are hashes not enough?

A SHA-256 hash proves that a file matches an expected reference. It does not prove that the original checkpoint, dataset, or dependency was safe. We still need origin verification, immutable model revisions, safe file formats and behavioral security evaluation.

Q4. Can the AI agent edit any Python file?

No. Our agent has an advisory role. Structured recommendations are limited to approved hyperparameters, and deterministic policy checks reject invalid values. Automatic code modification and production deployment are not permitted in the verified implementation.

Q5. How would you fix a real malicious dataset?

Do not automatically fine-tune again on the same dataset. Quarantine the suspicious input, record the evidence and affected runs, review its origin, replace or remediate it with human approval, and rerun validation/security checks before new training.

Q6. Which vulnerabilities have you actually discovered?

We have demonstrated detection and controlled recovery for training faults, plus artifact-integrity protection. We have **not** reported a confirmed CVE, poisoning incident, privacy leak, or adversarial model compromise from this project. The security-scan workflow is a planned next step.

Suggested one-line closing

"My contribution is connecting training observability, guarded AI recommendations and reproducible recovery. The next hardening milestone is adding real security scanning and fail-closed intake/deployment gates."

Documentation status: **v0.8.11** adds this interview-defense appendix to the existing 73-page handbook. Code baseline remains **v0.8.7**; this update introduces no additional security controls or test executions.

Phase 6B | Experiment Intelligence Dashboard

Cumulative handbook addendum - AutoTune AI v0.9.0

Architecture, implementation, security, test evidence, local deployment and next sprint.

Verification area	Current status
Windows CPU Hugging Face training	Previously user-verified
Windows CPU self-healing with injected failure	Previously user-verified; not natural overfitting
Windows localhost MLflow experiment logging	Previously user-verified: 3 run links
FastAPI read-only viewer	New code; 22 security/API tests passed
Complete Python suite	238 passing in build container
Next.js + TypeScript frontend	Source created; npm install/build NOT verified
Live Windows dashboard integration	Pending user acceptance

What changed?

AutoTune AI now includes a dedicated experiment-viewer API and a dark-mode web dashboard source. This layer reads MLflow tracking records but does not modify experiments, source code, checkpoints, models, training jobs or deployment state.

Why this architecture?

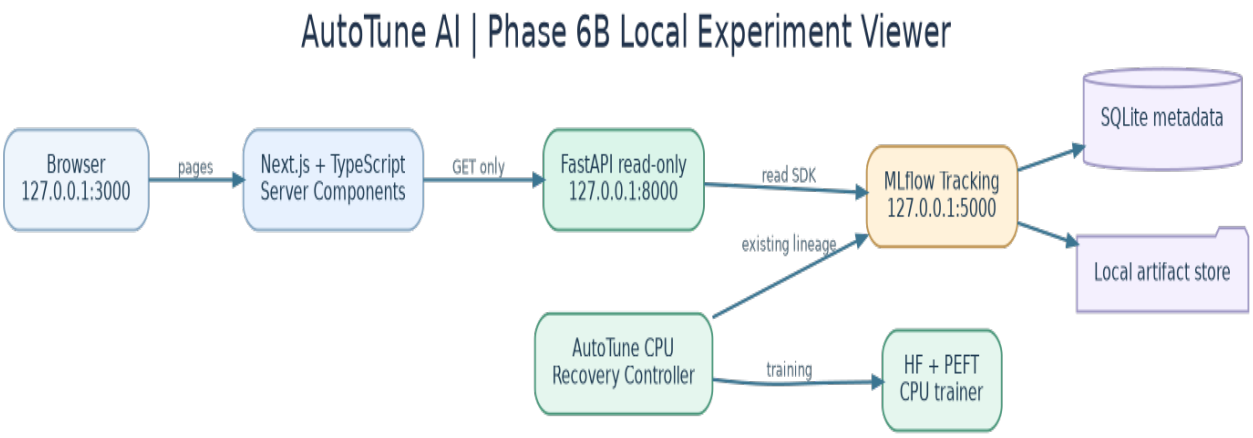
The existing training controller and its immutable local receipts remain the authority for training results. MLflow is a queryable observability view. The dashboard uses a narrow API boundary rather than granting a browser raw MLflow credentials or filesystem access.

Evidence limitation:

The 238 tests were executed against the source snapshot, with a fake MLflow client for the viewer. The new UI was not installed or built in the package-generation environment. Real MLflow-to-dashboard acceptance still needs to run on Windows.

System Architecture

The data path is one-directional from the browser to existing experiment records. The previous autonomous training controller remains separate.



Component connections

Component	Responsibility
Hugging Face + PEFT trainer	Performs real fine-tuning on CPU and saves validated checkpoint files.
AutoTune recovery controller	Controls stop/retry logic, applies allowed hyperparameters and records local receipts.
MLflow + SQLite	Stores parent-child experiment lineage, metrics and metadata.
FastAPI viewer :8000	Fixed local MLflow SDK client; exposes constrained, read-only JSON.
Next.js viewer :3000	Server-rendered experiment summary, run details and simple loss charts.

Read-only data boundary

The API intentionally has no POST/PUT/PATCH/DELETE training actions, arbitrary artifact retrieval, remote model import, filesystem path parameters, or commands that could execute models or Python code. The dashboard supports inspection only.

Data Contract and API Design

All API routes are confined to **GET** and use whitelisted, bounded inputs.

Route	Response / safeguard
/api/health	API status only; does not assert MLflow connectivity.
/api/experiments	Up to 100 experiment names and IDs.
/api/experiments/{id}/runs	1-100 latest runs; integer experiment ID.
/api/runs/{run_id}	Validated 32-character hex run identifier.
/api/runs/{run_id}/metrics/{metric}	Metric name allowlist, finite values, max 250 points.

Response filtering

Run responses include only selected scalar metric values, training configuration fields, run names, parent IDs, attempt roles, diagnoses and fault-injection flags. Arbitrary MLflow metadata and artifact paths are omitted; this reduces accidental exposure but cannot substitute for an authenticated service.

Lineage semantics

The MLflow parent is the AutoTune controller run; two child trials are associated by the `mlflow.parentRunId` tag. The UI lists all runs, shows trial diagnoses, and compares recorded validation losses. Differences measured on the tiny test set do not constitute significant quality improvements.

Failure behavior

Malformed IDs return HTTP 400, invalid limits return HTTP 422, unsupported write methods return HTTP 405, and internal MLflow exceptions return a sanitized HTTP 503. The underlying filesystem journal remains untouched.

Threat Boundaries and Security Controls

Threat	Mitigation (implemented in viewer)
SSRF via arbitrary MLflow server	Startup rejects any target except http://127.0.0.1:port without query, path or credentials.
Credential/path leakage	Allowlisted run fields, truncated safe strings, generic 503 errors.
Unexpected API write operations	Read-only GET endpoints; 405 for POST/DELETE.
Malformed run or metric input	Strict ID regular expressions and metric-name allowlist.
Unexpected browser origins	Explicit localhost:3000 and 127.0.0.1:3000 CORS origins.
Host-header abuse	TrustedHost allowlist for localhost and testserver.
Unbounded charts	250-point maximum returned for metric histories.

Not yet secure for multi-user deployments

The web UI, MLflow, and API do not implement authentication, fine-grained authorization, HTTPS termination, per-user scopes, request rate limiting or a formal vulnerability management gate. Keep them bound to loopback. Do not expose :3000, :5000 or :8000 to the internet or shared networks.

Vulnerabilities vs training anomalies

Overfitting and unstable gradients are monitored as training-health conditions, not cybersecurity vulnerabilities. Dataset poisoning, malicious model provenance, credential compromise and dependency CVEs require separate scanning and incident policy. This phase does not claim those scanners exist.

Windows Deployment and Verification

Open three independent PowerShell terminals from the existing project directory. Preserve all previous outputs and MLflow SQLite data.

Terminal A - MLflow

```
.\.venv\Scripts\mlflow.exe server --backend-store-uri sqlite:///mlflow.db
--host 127.0.0.1 --port 5000
```

Terminal B - FastAPI viewer

```
.\.venv\Scripts\python.exe -m pip install -r requirements-viewer.txt
$env:AUTOTUNE_MLFLOW_URI = "http://127.0.0.1:5000"
.\.venv\Scripts\python.exe -m uvicorn autotune.viewer_api:app
--host 127.0.0.1 --port 8000
```

Terminal C - Next.js + TypeScript

```
cd .\web
npm install
npm run typecheck
npm run build
npm run dev
```

Acceptance conditions

- GET http://127.0.0.1:8000/api/health responds with ready and mlflow: not probed.
- GET /api/experiments shows AutoTune-AI-CPU and its three prior run records.
- Next.js home, experiment and run detail pages render using the API, with validation-loss trend curves.
- Invalid paths and untrusted CORS origins are rejected; run details expose no secret fields.

Troubleshooting

If the viewer shows unavailable, check that MLflow server is running and that the backend is bound to port 8000. If npm install fails, check network access and Node version; no frontend build verification is assumed. Do not disable Windows security controls to solve dashboard issues.

Tests, Next Milestones, Decision Record

Test category	Outcome
Inherited ML/PEFT/recovery/MLflow Python tests	216 passing locally in build environment
New local API/SSRF/response-sanitization tests	22 passing
Total Python regression suite	238 passing
Node installation / Next.js build	Not completed in build environment (timeout)
Windows live MLflow -> FastAPI -> Next.js	Pending user-run acceptance

Next execution order

- **Finish Phase 6B acceptance:** Windows API + Next.js build and run-detail inspection using the real three-run MLflow experiment.
- **Security Sprint S0:** pip-audit/npm audit reporting, immutable Hugging Face model revisions, safe model serialization checks, dataset PII/security triage, deny/quarantine actions and test evidence.
- **Phase 6C:** richer metric streaming/refresh, comparative run views and authenticated controls only after explicit authorization design.
- **Phase 7-8:** Kubernetes/Kubeflow pipelines on CPU cluster and Katib HPO. Unsloth remains optional for future compatible GPU resources.

Preserved project truth

The user has already verified a real Tiny GPT-2 CPU LoRA run, fresh adapter reload, checkpoint continuation, a fault-injected two-attempt self-healing loop, and successful local MLflow tracking. The first stop was injected, and the observed validation difference was negligible. No deployment approval or security-scanner completion is claimed.

References

MLflow Tracking Client API: https://mlflow.org/docs/latest/api_reference/python_api/mlflow.client.html

Next.js App Router Data Fetching: https://nextjs.org/docs/app/getting-started/fetching-data

FastAPI CORS: https://fastapi.tiangolo.com/tutorial/cors/